

VIRTUAL LAB SETUP & WAZUH SIEM DEPLOYMENT

End-to-End Threat Detection, Custom Rule Engineering,
and Security Operations Center

CYBER SECURITY VIRTUAL LAB & SIEM DEPLOYMENT REPORT

Prepared by

Areeba Anwar

Cyber Security Student • Aspiring SOC Analyst

VMware Workstation Pro | Kali Linux | Wazuh SIEM | MITRE ATT&CK

Table of Contents

1. Objective
2. Executive Summary
3. Environment Detail
4. VMware Workstation Installation
5. VMware NAT Network Configuration
6. Kali Linux Installation & Configuration
7. Snapshot Creation
8. Wazuh Deployment
9. Wazuh Services Verification
10. Wazuh Dashboard Access
11. Connectivity Testing
12. Wazuh Agent Registration
13. Security Event Generation
14. File Integrity Monitoring (FIM) Testing & Verification
15. Network Reconnaissance Scan Against Wazuh Manager
16. Agent Deployment & Log Source Tracking
17. Custom Decoder & Detection Rule Engineering
18. Validation & Central Analysis
19. Role-Based Access Control (RBAC) Setup
20. Conclusion

1. Objective

Build and configure a secure cybersecurity virtual lab environment using VMware Workstation, Kali Linux (Red Team), and Wazuh SIEM (Blue Team) for safe security monitoring and threat detection practice.

■ Core Technical Goals

- Establish isolated NAT networking and verify seamless cross-platform connectivity between attacking and monitoring endpoints.
- Deploy and verify core Wazuh SIEM components including wazuh-manager, wazuh-indexer, and wazuh-dashboard.
- Register Wazuh agents to enable active endpoint monitoring, telemetry collection, and File Integrity Monitoring (FIM).
- Design XML-based custom decoders and detection rules (Rule 10003) mapped to the MITRE ATT&CK framework (e.g., T1110 Brute Force).
- Validate rule performance via wazuh-logtest and enforce Least Privilege Access using Role-Based Access Control (RBAC).

2. Executive Summary

This technical report documents the complete deployment, configuration, and validation of a hands-on cybersecurity lab environment. Using VMware Workstation Pro as the core virtualization platform, an isolated NAT network was constructed to host a Kali Linux attacker machine alongside a Wazuh SIEM appliance. Full operational viability was established through agent registration, system service verification, and cross-node connectivity testing.

The lab successfully demonstrated real-time threat monitoring and custom detection engineering capabilities. Key achievements include testing File Integrity Monitoring (FIM) on system files, capturing network reconnaissance scans (Nmap), and ingesting application logs via custom log source configurations. Custom XML decoders were engineered to parse unstructured logs into actionable fields, triggering custom severity alerts (Rule 10003) aligned with MITRE ATT&CK techniques. Finally, central visual dashboards were created for real-time analysis, and RBAC policies were established to ensure secure SOC analyst operations.

3. Environment Detail

Component	Purpose
VMware Workstation Pro	Virtualization Platform
Kali Linux	Attacker Machine (Red Team)
Wazuh OVA	SIEM & Security Monitoring (Blue Team)

4. VMware Workstation Installation

VMware Workstation Pro was installed successfully and verified.

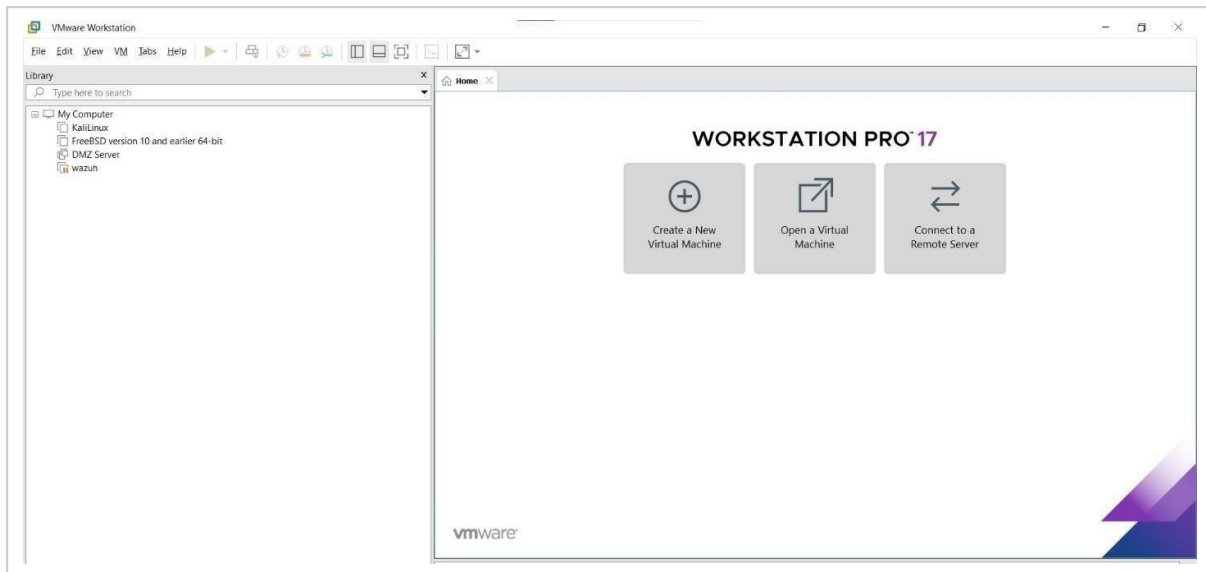


Fig. — VMware Workstation Home Screen

5. VMware NAT Network Configuration

A NAT network was configured to allow communication between virtual machines.

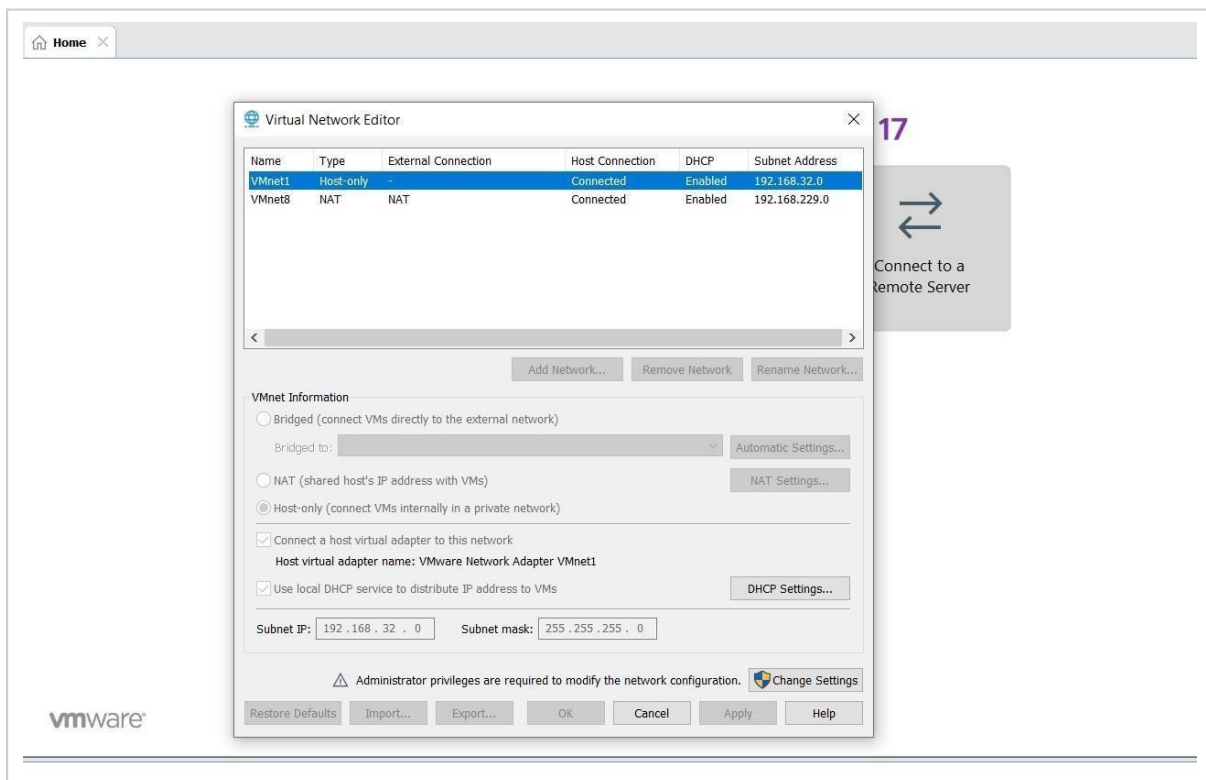


Fig. — VMnet8 NAT Configuration — Virtual Network Editor

6. Kali Linux Installation & Configuration

The Kali Linux virtual machine was imported successfully and connected to the NAT network.

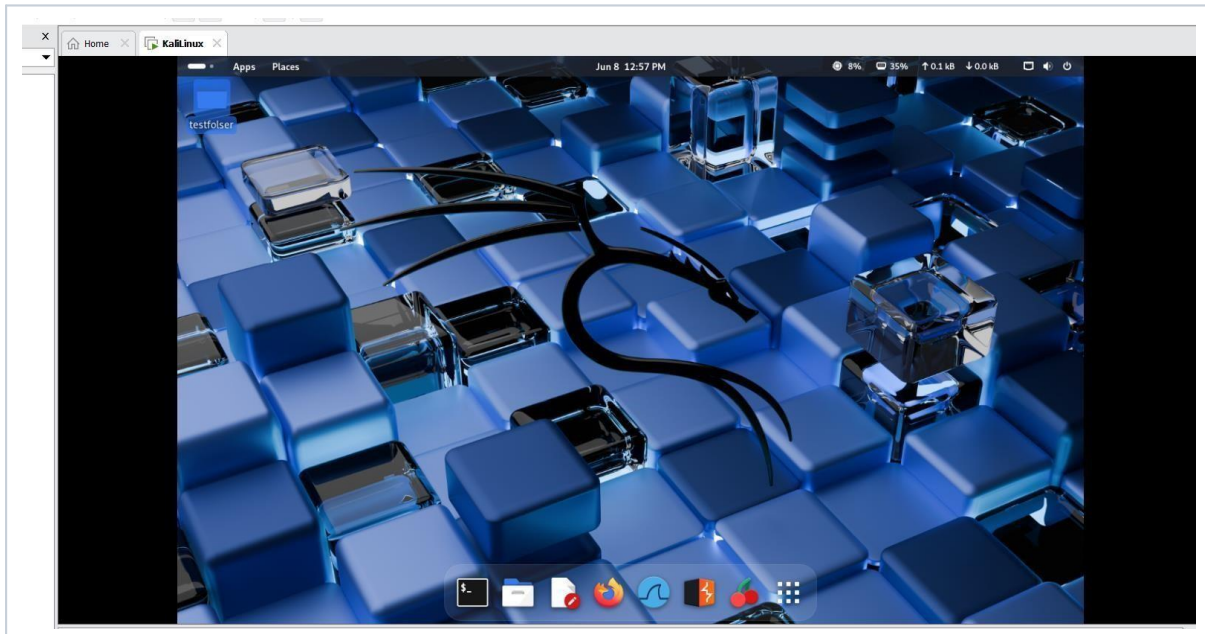


Fig. — Kali Linux Desktop

■ Verifying Kali Tools

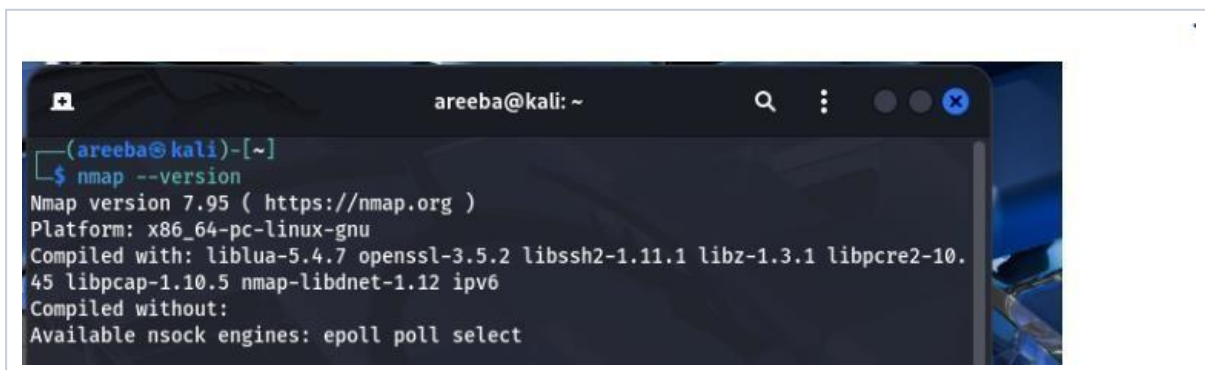


Fig. — nmap --version

```
areeba@kali: ~  
  
(areeba@kali)-[~]  
$ wireshark --version  
Wireshark 4.4.7.  
  
Copyright 1998-2025 Gerald Combs <gerald@wireshark.org> and contributors.  
Licensed under the terms of the GNU General Public License (version 2 or later).  
This is free software; see the file named COPYING in the distribution. There is  
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
  
Compiled (64-bit) using GCC 14.3.0, with GLib 2.84.4, with Qt 6.8.2, with  
libpcap, with POSIX capabilities (Linux), with libnl 3, with zlib 1.3.1, without  
zlib-ng, with PCRE2, with Lua 5.4.8, with GnuTLS 3.8.10 and PKCS #11 support,  
with Gcrypt 1.11.2, with Kerberos (MIT), with MaxMind, with nghttp2 1.64.0, with  
nghttp3 1.8.0, with brotli, with LZ4, with Zstandard, with Snappy, with libxml2  
2.14.5, with libsmi 0.4.8, with Minizip 1.3.1, with QtMultimedia, with QtDBus,  
without automatic updates, with binary plugins.  
  
Running on Linux 6.12.38+kali-amd64, with Intel(R) Core(TM) i5-7200U CPU @  
2.50GHz (with SSE4.2), with 3363 MB of physical memory, with GLib 2.84.4, with  
Qt 6.8.2, with libpcap 1.10.5 (with TPACKET_V3), with zlib 1.3.1, with PCRE2  
10.45 2025-02-05, with c-ares 1.34.5, with GnuTLS 3.8.9, with Gcrypt 1.11.2,  
with nghttp2 1.64.0, with nghttp3 1.8.0, with brotli 1.1.0, with LZ4 1.10.0,  
with Zstandard 1.5.7, with libsmi 0.4.8, with LC_TYPE=en_US.UTF-8, binary  
plugins supported.
```

Fig. — wireshark --version

```
(areeba@kali)-[~]  
$ burpsuite &  
[1] 6026  
  
(areeba@kali)-[~]  
$ [warning] /usr/bin/burpsuite: No JAVA_CMD set for run_java, falling back to JAVA_CMD = java  
Your JRE appears to be version 21.0.8 from Debian  
Burp has not been fully tested on this platform and you may experience problems.  
  
[1] + done      burpsuite  
(areeba@kali)-[~]
```

Fig. — burpsuite launch confirmation

```
(areeba@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:73:02:bb brd ff:ff:ff:ff:ff:ff  
    inet 192.168.220.128/24 brd 192.168.220.255 scope global dynamic noprefixroute eth0  
        valid_lft 1436sec preferred_lft 1436sec  
    inet6 fe80::efbb:f5cb:a9e0:5da2/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever
```

Fig. — ip a — network interface details

```
(areeba@kali)-[~]  
$ ifconfig eth0  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.220.128 netmask 255.255.255.0 broadcast 192.168.220.255  
    inet6 fe80::efbb:f5cb:a9e0:5da2 prefixlen 64 scopeid 0x20<link>  
    ether 00:0c:29:73:02:bb txqueuelen 1000 (Ethernet)  
    RX packets 328 bytes 28000 (27.3 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 162 bytes 25511 (24.9 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Fig. — `ifconfig eth0` — interface statistics

7. Snapshot Creation

A baseline snapshot of the Kali Linux VM was created to preserve a clean recovery state.

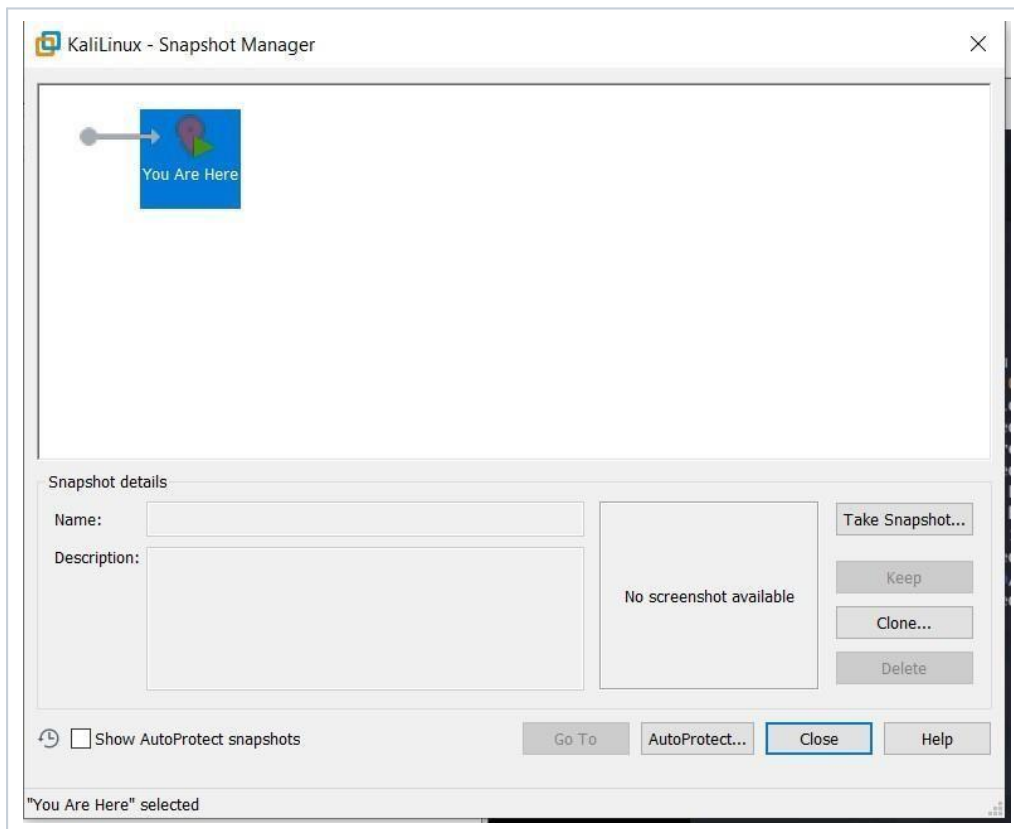


Fig. — Kali Linux Snapshot Manager

8. Wazuh Deployment

The Wazuh OVA appliance was imported and powered on successfully.

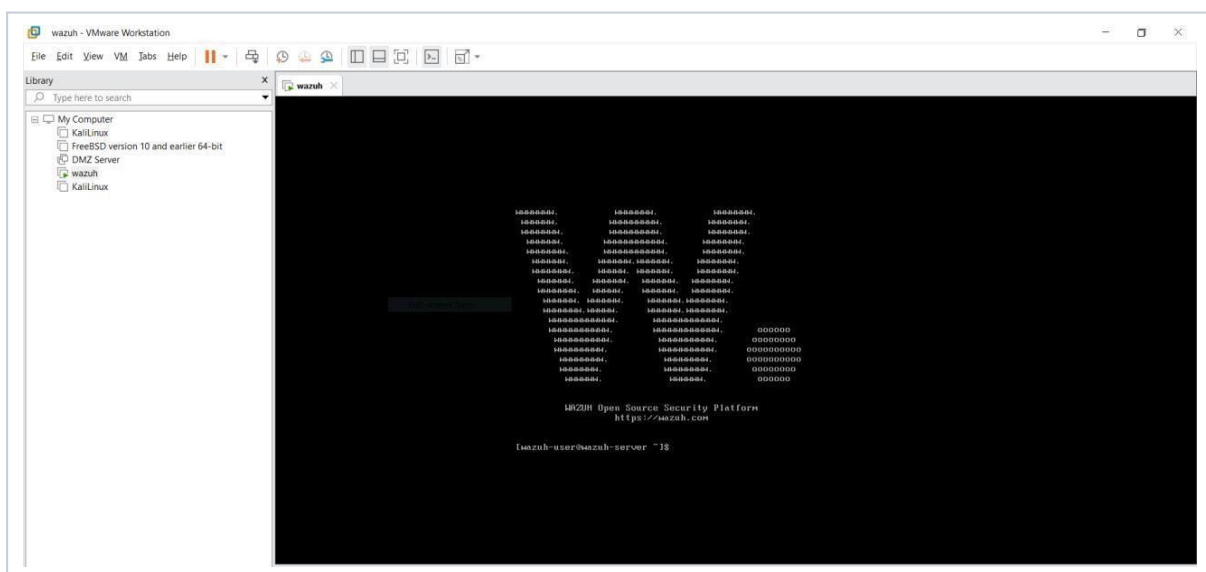
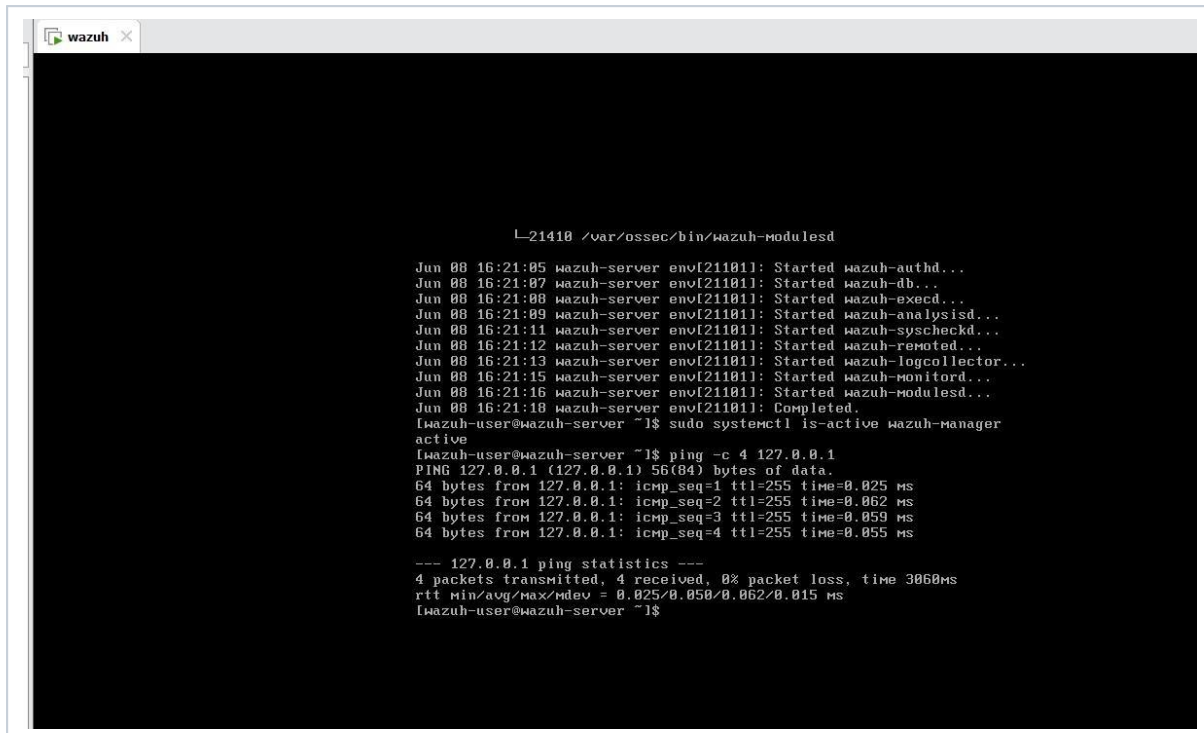


Fig. — Wazuh OVA boot sequence

■ Command Execution — Local Loopback Test

ping -c 4 127.0.0.1 — verifying wazuh-manager service state and loopback connectivity.



```

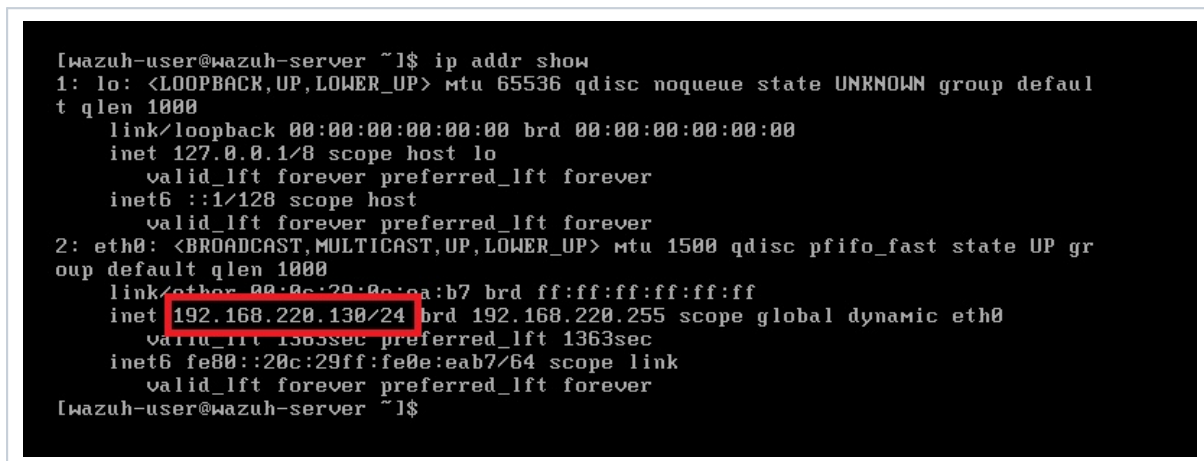
L-21410 /var/ossec/bin/wazuh-modulesd
Jun 08 16:21:05 wazuh-server env[21101]: Started wazuh-authd...
Jun 08 16:21:07 wazuh-server env[21101]: Started wazuh-db...
Jun 08 16:21:08 wazuh-server env[21101]: Started wazuh-execd...
Jun 08 16:21:09 wazuh-server env[21101]: Started wazuh-analysisd...
Jun 08 16:21:11 wazuh-server env[21101]: Started wazuh-syscheckd...
Jun 08 16:21:12 wazuh-server env[21101]: Started wazuh-remoted...
Jun 08 16:21:13 wazuh-server env[21101]: Started wazuh-logcollector...
Jun 08 16:21:15 wazuh-server env[21101]: Started wazuh-monitord...
Jun 08 16:21:16 wazuh-server env[21101]: Started wazuh-modulesd...
Jun 08 16:21:18 wazuh-server env[21101]: Completed.
[wazuh-user@wazuh-server ~]$ sudo systemctl is-active wazuh-manager
active
[wazuh-user@wazuh-server ~]$ ping -c 4 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=255 time=0.025 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=255 time=0.062 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=255 time=0.059 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=255 time=0.055 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3060ms
rtt min/avg/max/mdev = 0.025/0.050/0.062/0.015 ms
[wazuh-user@wazuh-server ~]$
  
```

Fig. — Wazuh manager services starting; loopback ping successful

■ Retrieving the Wazuh Server IP Address

ip addr show — used to identify the Wazuh appliance's assigned IP address on the NAT network.



```

[wazuh-user@wazuh-server ~]$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:20:0a:b7:ff brd ff:ff:ff:ff:ff:ff
    inet 192.168.220.130/24 brd 192.168.220.255 scope global dynamic eth0
        valid_lft 1363sec preferred_lft 1363sec
    inet6 fe80::20c:29ff:fe0e:eab7/64 scope link
        valid_lft forever preferred_lft forever
[wazuh-user@wazuh-server ~]$
  
```

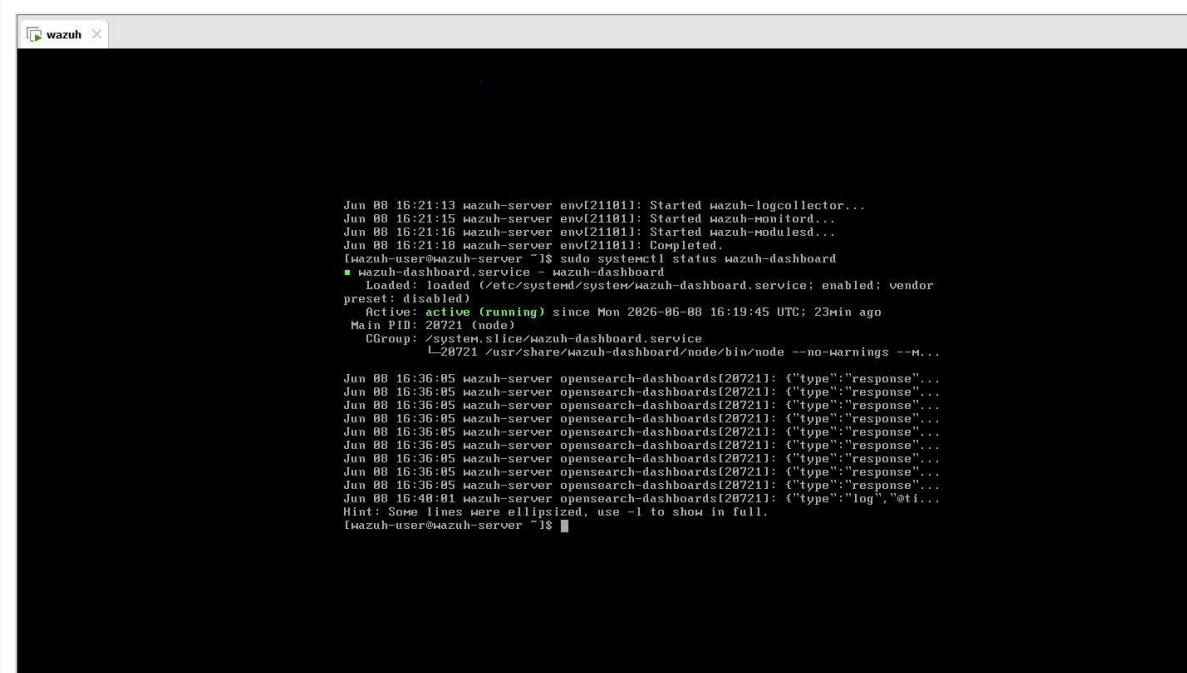
Fig. — Wazuh server IP address confirmed via ip addr show

9. Wazuh Services Verification

The following core services were verified as active and running:

- wazuh-manager
- wazuh-indexer
- wazuh-dashboard

■ Service Status — wazuh-dashboard

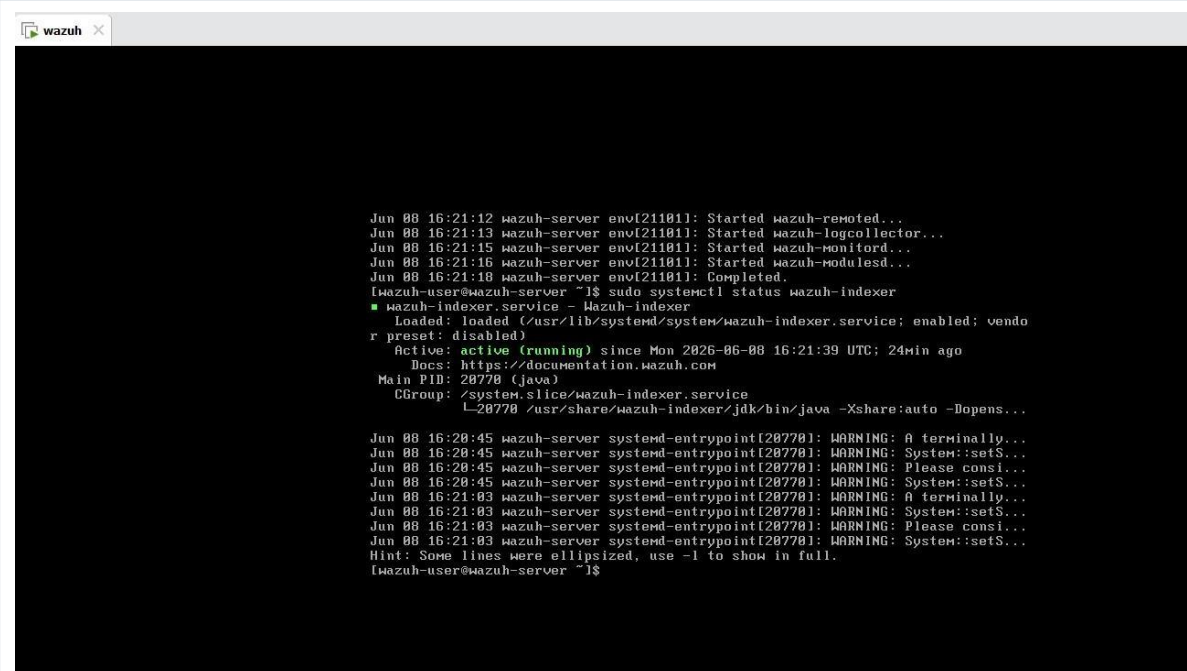


```
Jun 08 16:21:13 wazuh-server env[21101]: Started wazuh-logcollector...
Jun 08 16:21:15 wazuh-server env[21101]: Started wazuh-monitor...
Jun 08 16:21:16 wazuh-server env[21101]: Started wazuh-modulesd...
Jun 08 16:21:18 wazuh-server env[21101]: Completed.
[wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-dashboard
■ wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; vendor
   preset: disabled)
   Active: active (running) since Mon 2025-06-08 16:19:45 UTC; 23min ago
     Main PID: 28721 (node)
    CGroup: /system.slice/wazuh-dashboard.service
            └─28721 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --H...

Jun 08 16:36:05 wazuh-server opensearch-dashboards[28721]: {"type":"response"...
Jun 08 16:36:05 wazuh-server opensearch-dashboards[28721]: {"type":"response"...
Jun 08 16:36:05 wazuh-server opensearch-dashboards[28721]: {"type":"response"...
Jun 08 16:36:05 wazuh-server opensearch-dashboards[28721]: {"type":"response"...
Jun 08 16:36:05 wazuh-server opensearch-dashboards[28721]: {"type":"response"...
Jun 08 16:36:05 wazuh-server opensearch-dashboards[28721]: {"type":"response"...
Jun 08 16:36:05 wazuh-server opensearch-dashboards[28721]: {"type":"response"...
Jun 08 16:36:05 wazuh-server opensearch-dashboards[28721]: {"type":"response"...
Jun 08 16:40:01 wazuh-server opensearch-dashboards[28721]: {"type":"log","@t:...
Hint: Some lines were ellipsized, use -l to show in full.
[wazuh-user@wazuh-server ~]$
```

Fig. — wazuh-dashboard service status: active (running)

■ Service Status — wazuh-indexer



```
Jun 08 16:21:12 wazuh-server env[21101]: Started wazuh-remoted...
Jun 08 16:21:13 wazuh-server env[21101]: Started wazuh-logcollector...
Jun 08 16:21:15 wazuh-server env[21101]: Started wazuh-monitor...
Jun 08 16:21:16 wazuh-server env[21101]: Started wazuh-modulesd...
Jun 08 16:21:18 wazuh-server env[21101]: Completed.
[wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-indexer
■ wazuh-indexer.service - Wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; vendo
   r preset: disabled)
   Active: active (running) since Mon 2025-06-08 16:21:39 UTC; 24min ago
     Docs: https://documentation.wazuh.com
     Main PID: 28770 (java)
    CGroup: /system.slice/wazuh-indexer.service
            └─28770 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopens...

Jun 08 16:20:45 wazuh-server systemd-entrypoint[28770]: WARNING: A terminally...
Jun 08 16:20:45 wazuh-server systemd-entrypoint[28770]: WARNING: System: setS...
Jun 08 16:20:45 wazuh-server systemd-entrypoint[28770]: WARNING: Please consi...
Jun 08 16:20:45 wazuh-server systemd-entrypoint[28770]: WARNING: System: setS...
Jun 08 16:21:03 wazuh-server systemd-entrypoint[28770]: WARNING: A terminally...
Jun 08 16:21:03 wazuh-server systemd-entrypoint[28770]: WARNING: System: setS...
Jun 08 16:21:03 wazuh-server systemd-entrypoint[28770]: WARNING: Please consi...
Jun 08 16:21:03 wazuh-server systemd-entrypoint[28770]: WARNING: System: setS...
Hint: Some lines were ellipsized, use -l to show in full.
[wazuh-user@wazuh-server ~]$
```

Fig. — wazuh-indexer service status: active (running)

■ Service Status — wazuh-manager

```
└─21166 /var/ossec/framework/python/bin/python3 /var/ossec/api/scr...
└─21169 /var/ossec/framework/python/bin/python3 /var/ossec/api/scr...
└─21212 /var/ossec/bin/wazuh-authd
└─21228 /var/ossec/bin/wazuh-db
└─21254 /var/ossec/bin/wazuh-execd
└─21278 /var/ossec/bin/wazuh-analysisd
└─21282 /var/ossec/bin/wazuh-syscheckd
└─21332 /var/ossec/bin/wazuh-remoted
└─21369 /var/ossec/bin/wazuh-logcollector
└─21388 /var/ossec/bin/wazuh-monitord
└─21418 /var/ossec/bin/wazuh-modulesd

Jun 08 16:21:05 wazuh-server env[21101]: Started wazuh-authd...
Jun 08 16:21:07 wazuh-server env[21101]: Started wazuh-db...
Jun 08 16:21:08 wazuh-server env[21101]: Started wazuh-execd...
Jun 08 16:21:09 wazuh-server env[21101]: Started wazuh-analysisd...
Jun 08 16:21:11 wazuh-server env[21101]: Started wazuh-syscheckd...
Jun 08 16:21:12 wazuh-server env[21101]: Started wazuh-remoted...
Jun 08 16:21:13 wazuh-server env[21101]: Started wazuh-logcollector...
Jun 08 16:21:15 wazuh-server env[21101]: Started wazuh-monitord...
Jun 08 16:21:16 wazuh-server env[21101]: Started wazuh-modulesd...
Jun 08 16:21:18 wazuh-server env[21101]: Completed.
[wazuh-user@wazuh-server ~]# sudo systemctl is-active wazuh-manager
active
[wazuh-user@wazuh-server ~]#
```

Fig. — wazuh-manager process tree and startup log; service status: active (running)

10. Wazuh Dashboard Access

The Wazuh web dashboard was accessed successfully through the browser.

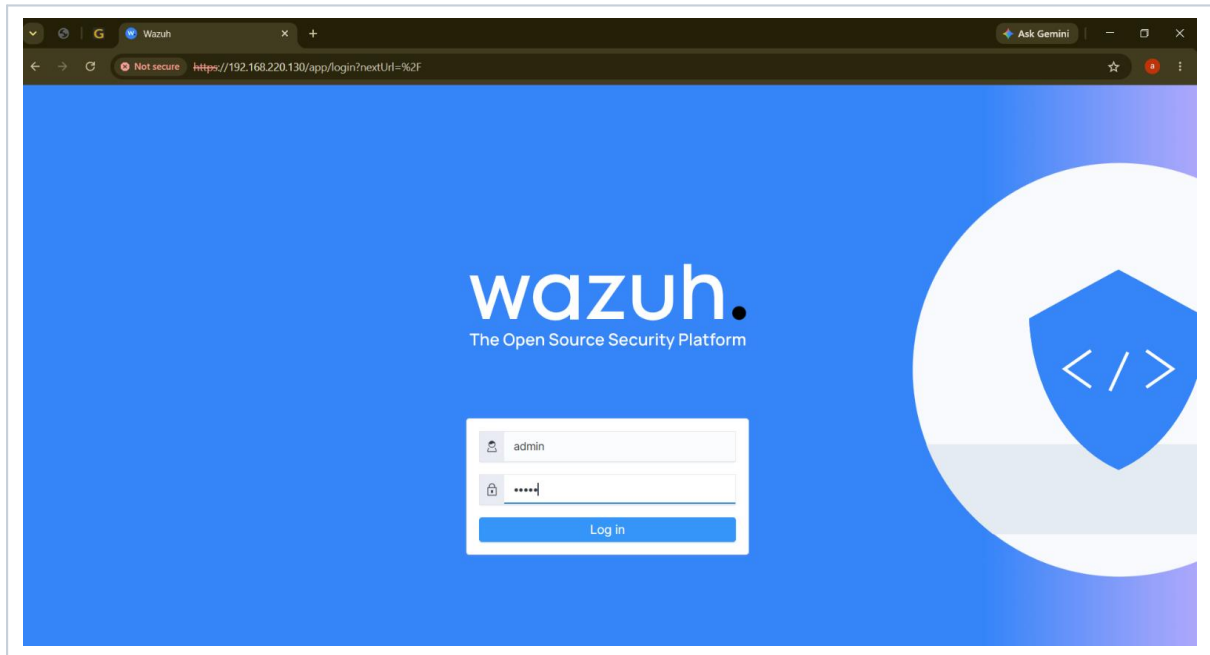


Fig. — Wazuh dashboard login page

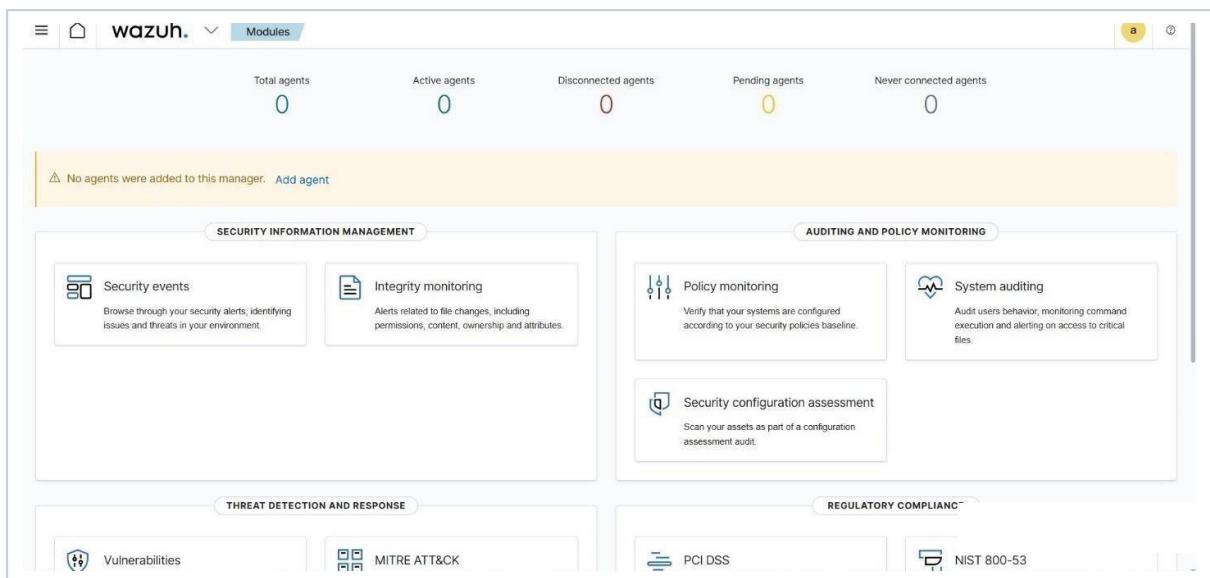
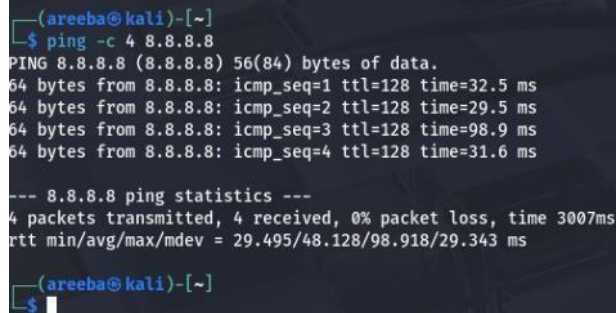


Fig. — Wazuh dashboard — modules overview

11. Connectivity Testing

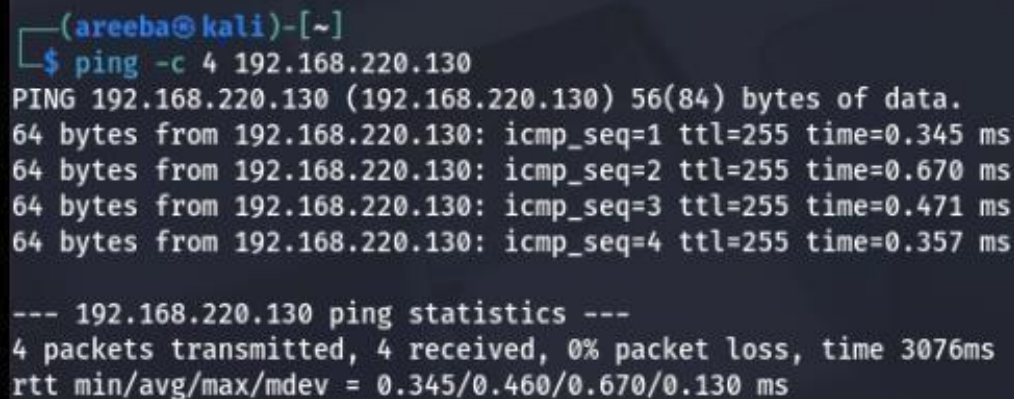
Connectivity between Kali Linux and the Wazuh appliance was verified using bidirectional ping tests.

■ From the Kali Terminal



```
(areeba@kali)-[~]  
$ ping -c 4 8.8.8.8  
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=32.5 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=29.5 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=98.9 ms  
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=31.6 ms  
  
--- 8.8.8.8 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3007ms  
rtt min/avg/max/mdev = 29.495/48.128/98.918/29.343 ms  
  
(areeba@kali)-[~]  
$
```

Fig. — ping -c 4 8.8.8.8 — external connectivity confirmed



```
(areeba@kali)-[~]  
$ ping -c 4 192.168.220.130  
PING 192.168.220.130 (192.168.220.130) 56(84) bytes of data:  
64 bytes from 192.168.220.130: icmp_seq=1 ttl=255 time=0.345 ms  
64 bytes from 192.168.220.130: icmp_seq=2 ttl=255 time=0.670 ms  
64 bytes from 192.168.220.130: icmp_seq=3 ttl=255 time=0.471 ms  
64 bytes from 192.168.220.130: icmp_seq=4 ttl=255 time=0.357 ms  
  
--- 192.168.220.130 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3076ms  
rtt min/avg/max/mdev = 0.345/0.460/0.670/0.130 ms
```

Fig. — ping -c 4 192.168.220.130 — Wazuh manager reachable

■ From the Wazuh Terminal

```
[wazuh-user@wazuh-server ~]$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=1042 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=30.5 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=35.3 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 3 received, 25% packet loss, time 3053ms
rtt min/avg/max/mdev = 30.588/369.549/1042.738/476.020 ms, pipe 2
[wazuh-user@wazuh-server ~]$ ping -c 4 198.168.220.129
PING 198.168.220.129 (198.168.220.129) 56(84) bytes of data.

--- 198.168.220.129 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3091ms

[wazuh-user@wazuh-server ~]$
```

Fig. — ping -c 4 8.8.8.8 — outbound connectivity from Wazuh manager

■ Kali Agent Connection

```
[wazuh-user@wazuh-server ~]$ ping -c 4 192.168.220.128
PING 192.168.220.128 (192.168.220.128) 56(84) bytes of data.
64 bytes from 192.168.220.128: icmp_seq=1 ttl=64 time=0.346 ms
64 bytes from 192.168.220.128: icmp_seq=2 ttl=64 time=0.475 ms
64 bytes from 192.168.220.128: icmp_seq=3 ttl=64 time=0.422 ms
64 bytes from 192.168.220.128: icmp_seq=4 ttl=64 time=0.501 ms

--- 192.168.220.128 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3078ms
rtt min/avg/max/mdev = 0.346/0.436/0.501/0.059 ms
[wazuh-user@wazuh-server ~]$
```

Fig. — ping -c 4 192.168.220.128 — Kali endpoint reachable from Wazuh manager

■ Active Wazuh Agent

```
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: wazuh-server (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: kali, IP: any, Active

List of agentless devices:

[wazuh-user@wazuh-server ~]$
```

Fig. — agent_control -l — confirming active registered agent

12. Wazuh Agent Registration

The Wazuh agent was installed on Kali Linux and successfully registered with the Wazuh Manager.

■ Adding the Wazuh Repository on Kali

```
(areeba@kali)-[~]
└─$ curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --dearmor | sudo tee /usr/share/keyrings/wazuh.gpg > /dev/null

(areeba@kali)-[~]
└─$ echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee /etc/apt/sources.list.d/wazuh.list
tee: /etc/apt/sources.list.d/wazuh.list: No such file or directory
deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main

(areeba@kali)-[~]
└─$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
Hit:2 https://packages.wazuh.com/4.x/apt stable InRelease
2955 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Fig. — Wazuh GPG key import and repository configuration

■ Wazuh Agent Installation

```
(areeba@kali)-[~]
└─$ sudo WAZUH_MANAGER='192.168.220.130' apt-get install wazuh-agent -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following packages will be upgraded:
  wazuh-agent
1 upgraded, 0 newly installed, 0 to remove and 2954 not upgraded.
Need to get 13.2 MB of archives.
After this operation, 18.0 MB of additional disk space will be used.
Get:1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-agent amd64 4.14.5-1 [13.2 MB]
Fetched 13.2 MB in 39s (341 kB/s)
Preconfiguring packages ...
(Reading database ... 549010 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.14.5-1_amd64.deb ...
Unpacking wazuh-agent (4.14.5-1) over (4.7.0-1) ...
Setting up wazuh-agent (4.14.5-1) ...

Configuration file '/etc/systemd/system/wazuh-agent.service'
==> Deleted (by you or by a script) since installation.
==> Package distributor has shipped an updated version.
What would you like to do about it ? Your options are:
  Y or I : install the package maintainer's version
  N or O : keep your currently-installed version
  D      : show the differences between the versions
  Z      : start a shell to examine the situation
The default action is to keep your current version.
*** wazuh-agent.service (Y/I/N/O/D/Z) [default=N] ? Y
Installing new version of config file /etc/systemd/system/wazuh-agent.service ...
```

Fig. — apt-get install wazuh-agent


```
(areeba@kali)-[~]
$ sudo systemctl status wazuh-agent
[sudo] password for areeba:
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-06-13 15:58:58 EDT; 1h 8min ago
  Invocation: 1063e7e973dc49c9932b9a7112605234
     Tasks: 28 (limit: 3876)
    Memory: 9M (peak: 339.7M, swap: 16.8M, swap peak: 17M)
       CPU: 40.765s
    CGroup: /system.slice/wazuh-agent.service
            └─9163 /var/ossec/bin/wazuh-execd
              └─9182 /var/ossec/bin/wazuh-agentd
                └─9201 /var/ossec/bin/wazuh-syscheckd
                  └─9220 /var/ossec/bin/wazuh-logcollector
                    └─9243 /var/ossec/bin/wazuh-modulesd

Jun 13 15:58:50 kali systemd[1]: Starting wazuh-agent.service - Wazuh agent...
Jun 13 15:58:50 kali env[9133]: Starting Wazuh v4.14.5...
Jun 13 15:58:52 kali env[9133]: Started wazuh-execd...
Jun 13 15:58:53 kali env[9133]: Started wazuh-agentd...
Jun 13 15:58:54 kali env[9133]: Started wazuh-syscheckd...
Jun 13 15:58:55 kali env[9133]: Started wazuh-logcollector...
Jun 13 15:58:56 kali env[9133]: Started wazuh-modulesd...
Jun 13 15:58:58 kali env[9133]: Completed.
Jun 13 15:58:58 kali systemd[1]: Started wazuh-agent.service - Wazuh agent.
```

Fig. — systemctl status wazuh-agent — active (running)

■ Agent Registration Confirmed in Dashboard

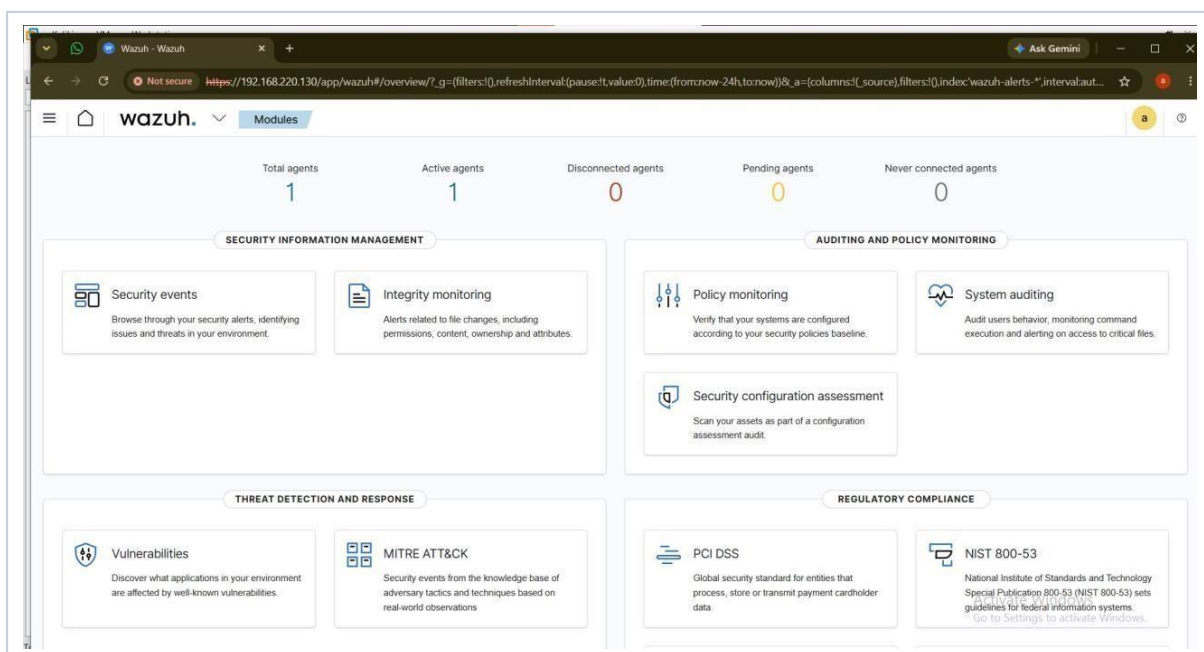


Fig. — Agent count updated to 1 active agent on the dashboard

■ Agent Configuration



```
areeba@kali: ~  
(areeba@kali)-[~]  
$ sudo cat /var/ossec/etc/ossec.conf | grep -A2 "<server>"  
[sudo] password for areeba:  
  <server>  
    <address>192.168.220.129</address>  
    <port>1514</port>  
(areeba@kali)-[~]  
$
```

Fig. — ossec.conf — server address and port configuration

13. Security Event Generation

A test event was generated and successfully detected by Wazuh — a failed SSH login brute-force simulation.

■ SSH Login (Failed Attempts)

```
(areeba@kali)-[~]
└─$ for i in 1 2 3 4 5; do ssh wronguser@192.168.220.130 2>/dev/null || true; done
The authenticity of host '192.168.220.130 (192.168.220.130)' can't be established.
ECDSA key fingerprint is SHA256:r9eJRrxYuLMuFq1HokQ0msKm028Gua0eZl+FA2DJb9U.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
wronguser@192.168.220.130's password:
wronguser@192.168.220.130's password:
wronguser@192.168.220.130's password:
wronguser@192.168.220.130's password:
wronguser@192.168.220.130's password:
wronguser@192.168.220.130's password:
```

Fig. — Simulated failed SSH login attempts against the Wazuh manager

■ Security Event Visible in Dashboard

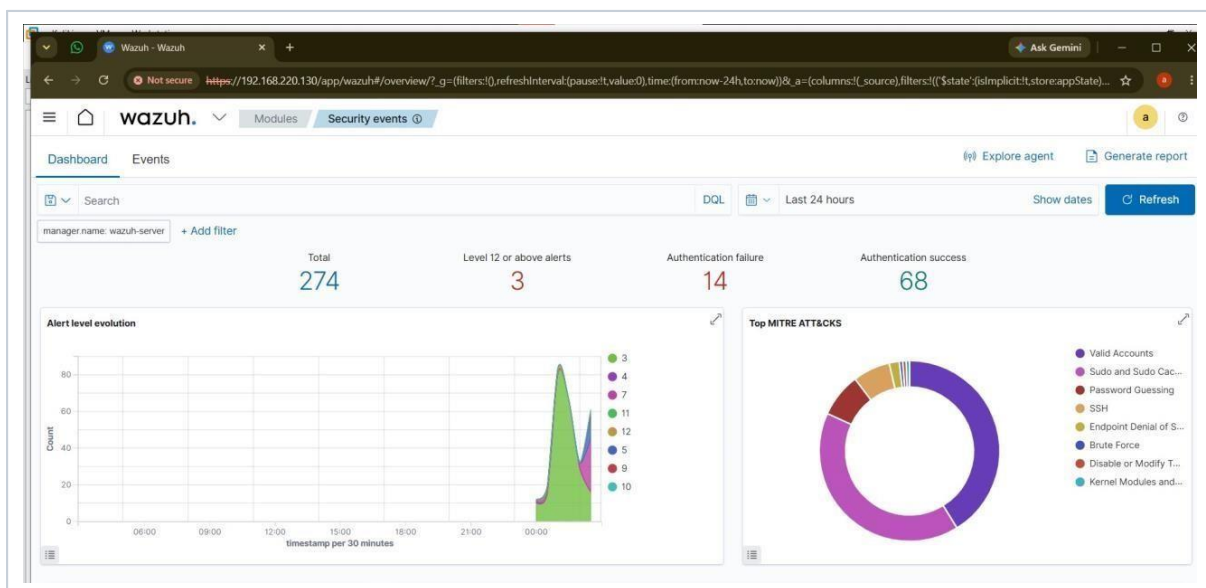


Fig. — Alert level evolution and Top MITRE ATT&CK; techniques

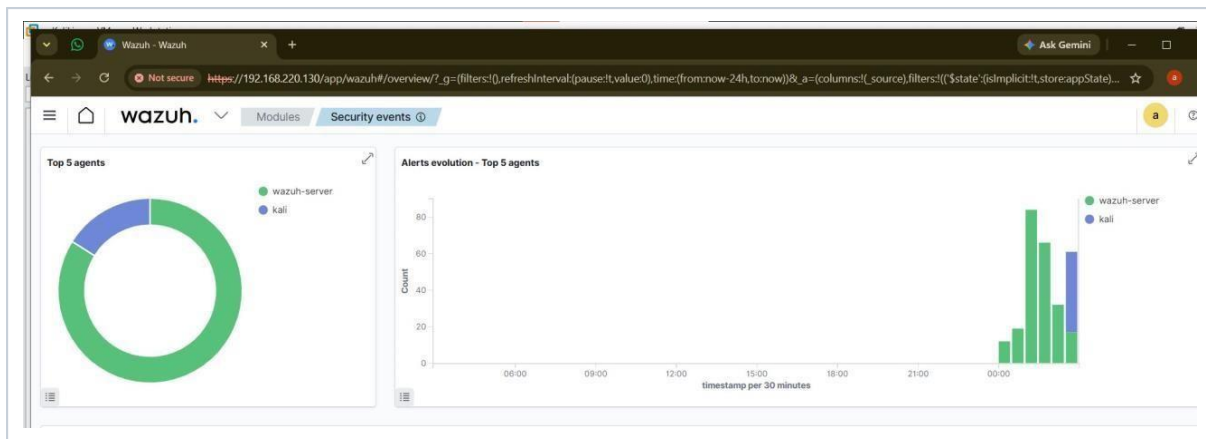


Fig. — Top 5 agents and alert evolution trend

The figure displays the Wazuh Security Alerts table, showing a list of failed SSH login attempts. The table includes columns for Time, Agent, Agent name, Technique(s), Tactic(s), Description, Level, and Rule ID. The alerts are filtered for 'ssh: Attempt to login using a non-existent user' and 'ssh: brute force trying to get access to the system. Non-existent user.' The table shows several instances of these alerts, with Rule IDs 5710, 5503, and 5712.

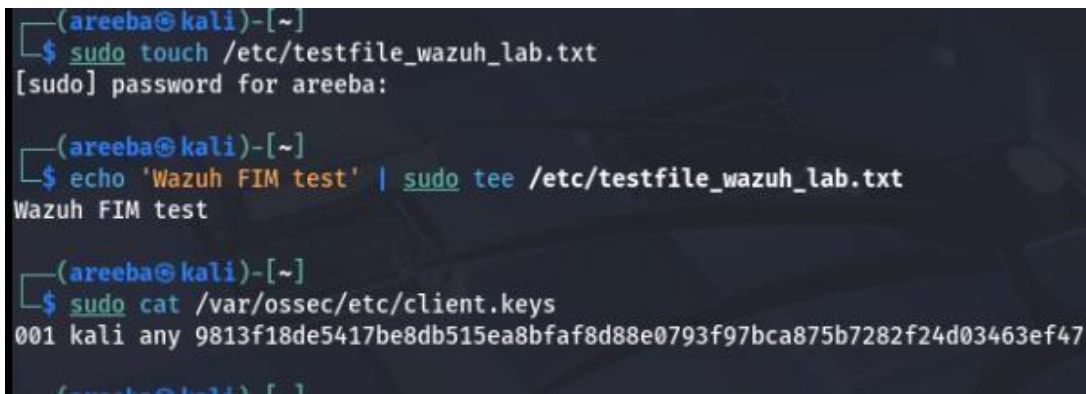
Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Jun 14, 2026 @ 02:54:39.047	000	wazuh-server	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
Jun 14, 2026 @ 02:54:34.948	000	wazuh-server	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
Jun 14, 2026 @ 02:53:55.006	000	wazuh-server	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
Jun 14, 2026 @ 02:53:52.906	000	wazuh-server	T1110.001	Credential Access	PAM: User login failed.	5	5503
Jun 14, 2026 @ 02:53:16.861	000	wazuh-server	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
Jun 14, 2026 @ 02:53:14.858	000	wazuh-server	T1110	Credential Access	sshd: brute force trying to get access to the system. Non-existent user.	10	5712
Jun 14, 2026 @ 02:53:06.891	000	wazuh-server	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710

Fig. — Security Alerts table — failed SSH login attempts (Rule IDs 5710, 5503, 5712)

14. File Integrity Monitoring (FIM) Testing & Verification

To evaluate Wazuh's File Integrity Monitoring (Syscheck) capabilities, a file modification test was performed on the Kali Linux endpoint: a monitored system file was created at `/etc/testfile_wazuh_lab.txt` using root privileges, populated with test data, and local agent registration keys were verified.

■ Executing FIM Test on the Kali Agent

A terminal window on a Kali Linux machine showing the execution of three commands. The first command creates a file, the second writes test data to it, and the third displays the agent's registration key from the Wazuh configuration file.

```
(areeba@kali)-[~]
└─$ sudo touch /etc/testfile_wazuh_lab.txt
[sudo] password for areeba:

(areeba@kali)-[~]
└─$ echo 'Wazuh FIM test' | sudo tee /etc/testfile_wazuh_lab.txt
Wazuh FIM test

(areeba@kali)-[~]
└─$ sudo cat /var/ossec/etc/client.keys
001 kali any 9813f18de5417be8db515ea8bfaf8d88e0793f97bca875b7282f24d03463ef47
```

Fig. — Test file creation and agent key verification

The Syscheck daemon detected the file creation in real time and generated a security alert, forwarded to the Wazuh Manager and displayed under the Integrity Monitoring dashboard.

Agent Name	kali
File Path	/etc/testfile_wazuh_lab.txt
Event Type	added
Rule Description	File added to the system (Rule ID: 554, Level: 5)

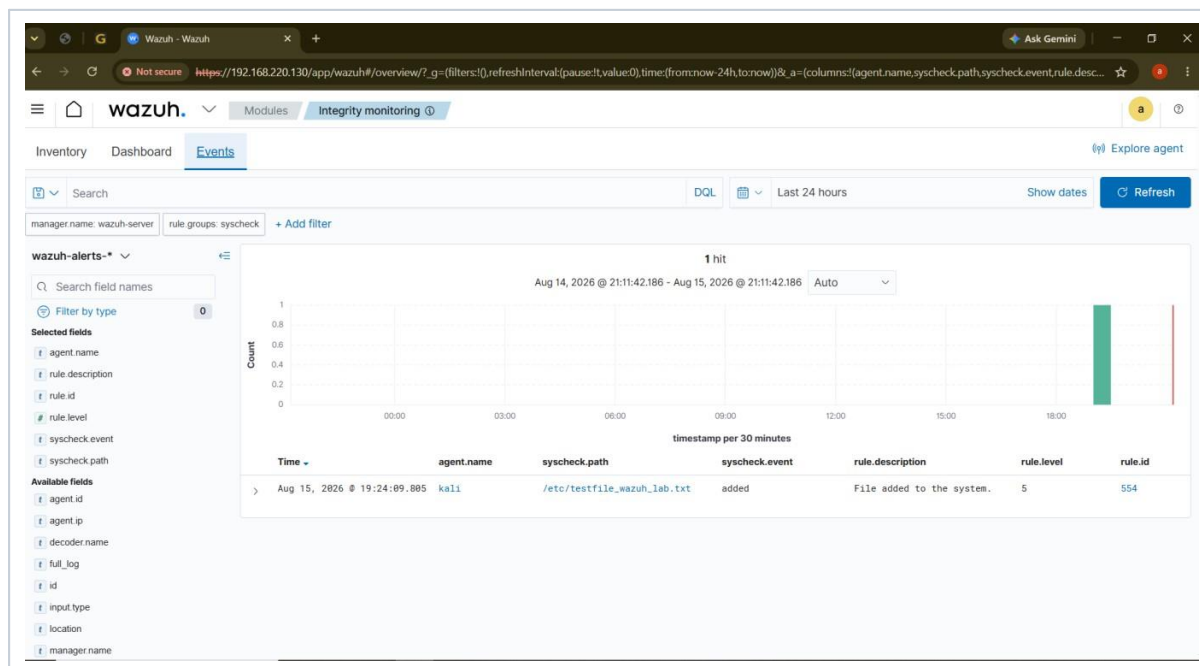


Fig. — Integrity Monitoring dashboard — file addition event detected

15. Network Reconnaissance Scan Against Wazuh Manager

To simulate network reconnaissance and test port visibility, an active port scan was executed from the Kali Linux host targeting the Wazuh Manager.

sudo nmap -sS -sV -O 192.168.220.130 — TCP SYN stealth scan, service version detection, and OS fingerprinting.

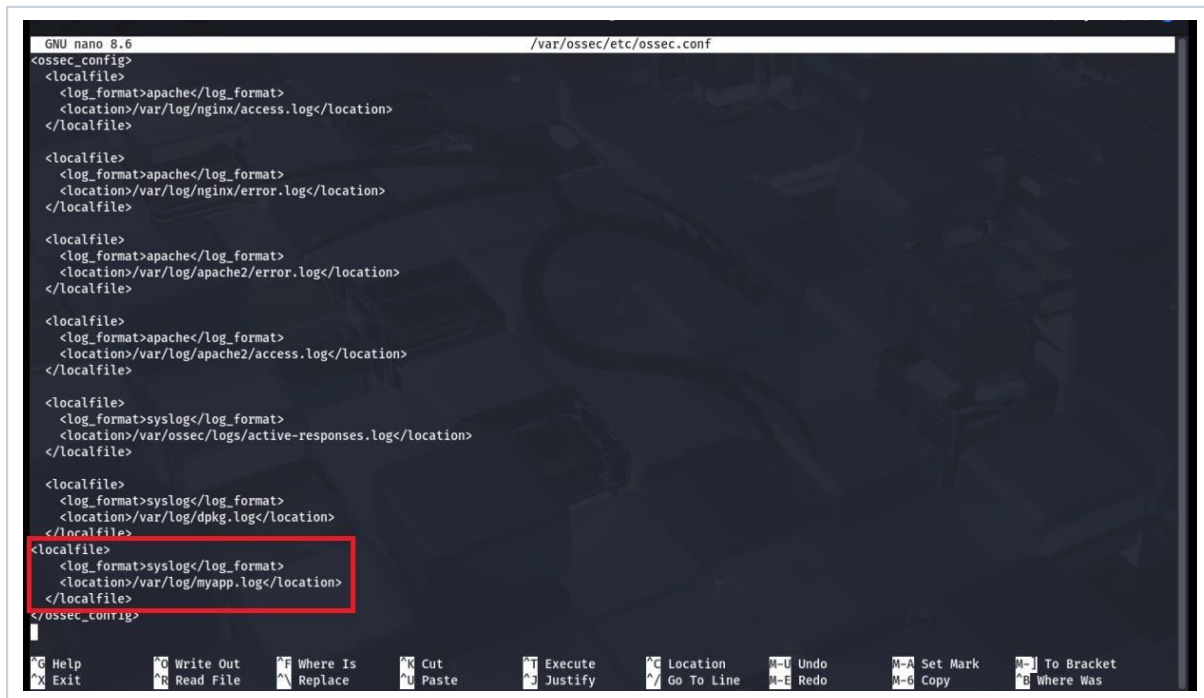
```
(areeba@kali)~$ sudo nmap -sS -sV -O 192.168.220.130
[sudo] password for areeba:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-15 12:15 EDT
Nmap scan report for 192.168.220.130
Host is up (0.0012s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4 (protocol 2.0)
111/tcp   open  rpcbind  2-4 (RPC #100000)
443/tcp   open  ssl/https
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port443-TCP:V=7.95%T=SSL%I=7%D=8/15%Time=6A8090CE%P=x86_64-pc-linux-gnu
SF:~r(GetRequest,15C,"HTTP/1.1\x20302\x20Found\r\nlocation:\x20/app/login
SF:~?r\nosd-name:\x20wazuh-server\r\nx-frame-options:\x20sameorigin\r\nca
SF:~che-control:\x20private,\x20no-cache,\x20no-store,\x20must-revalidate\r
SF:~nset-cookie:\x20security_authentication;\x20Max-Age=0;\x20Expires=Thu
SF:~\x2001\x20Jan\x201970\x2000:00:00\x20GMT;\x20Secure;\x20HttpOnly;\x20P
SF:~ath=/\r\ncontent-length:\x200\r\nDate:\x20Sat,\x2015\x20Aug\x202026\x20
SF:~16:16:13\x20GMT\r\nConnection:\x20close\r\n\r\n")~r(HTTPOptions,143,"HT
SF:~P/1.1\x20404\x20Not\x20Found\r\nosd-name:\x20wazuh-server\r\nx-frame-
SF:~options:\x20sameorigin\r\ncontent-type:\x20application/json;\x20charset
SF:~utf-8\r\nx-cache-control:\x20private,\x20no-cache,\x20no-store,\x20must-
SF:~revalidate\r\ncontent-length:\x2060\r\nDate:\x20Sat,\x2015\x20Aug\x2020
SF:~26\x2016:16:14\x20GMT\r\nConnection:\x20close\r\n\r\n{"statusCode":40
SF:~4,"error":{"Not\x20Found"},"message":{"Not\x20Found"}}~r(FourOhFo
SF:~urRequest,1C9,"HTTP/1.1\x20401\x20Unauthorized\r\nosd-name:\x20wazuh-s
SF:~erver\r\nx-frame-options:\x20sameorigin\r\ncontent-type:\x20application
SF:~/json;\x20charset=utf-8\r\nx-cache-control:\x20private,\x20no-cache,\x20n
SF:~o-store,\x20must-revalidate\r\nset-cookie:\x20security_authentication;
SF:~\x20Max-Age=0;\x20Expires=Thu,\x2001\x20Jan\x201970\x2000:00:00\x20GMT;
SF:~\x20Secure;\x20HttpOnly;\x20Path=/\r\ncontent-length:\x2077\r\nDate:\x2
SF:~0Sat,\x2015\x20Aug\x202026\x2016:16:14\x20GMT\r\nConnection:\x20close\r
SF:~\n\r\n{"statusCode":401,"error":{"Unauthorized"},"message":{"Auth
SF:~entication\x20required"}}~r(RTSPRequest,143,"HTTP/1.1\x20404\x20Not
SF:~\x20Found\r\nosd-name:\x20wazuh-server\r\nx-frame-options:\x20sameorigin
SF:~\r\ncontent-type:\x20application/json;\x20charset=utf-8\r\nx-cache-contro
SF:~l:\x20private,\x20no-cache,\x20no-store,\x20must-revalidate\r\ncontent-
```

Fig. — Nmap reconnaissance scan results against the Wazuh manager

16. Agent Deployment & Log Source Tracking

■ Custom Log File Configuration

A `<localfile>` block pointing to application log paths was added inside `/var/ossec/etc/ossec.conf` on the endpoint, instructing the Wazuh Agent to monitor application logs and forward events to the manager.



```
GNU nano 8.6 /var/ossec/etc/ossec.conf
<ossec_config>
  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/nginx/access.log</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/nginx/error.log</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/apache2/error.log</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/apache2/access.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/dpkg.log</location>
  </localfile>
  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/myapp.log</location>
  </localfile>
</ossec_config>
```

Fig. — `ossec.conf` — custom `localfile` log source definitions

■ Deploying & Enrolling the Endpoint

The `wazuh-agent` package was installed on Kali Linux and connection status verified, establishing endpoint-to-manager telemetry collection.

wazuh. ▾

Agents

Deploy new agent

✓

Select the package to download and install on your system:

 LINUX

☐ RPM amd64 ☐ RPM aarch64
☒ DEB amd64 ☐ DEB aarch64

 WINDOWS

☐ MSI 32/64 bits

 macOS

☐ Intel
☐ Apple silicon

① For additional systems and architectures, please check our documentation [🔗](#).

✓

Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FDQN).

Assign a server address: [?](#)

192.168.220.128

Fig. — Deploy new agent wizard — package selection and server address

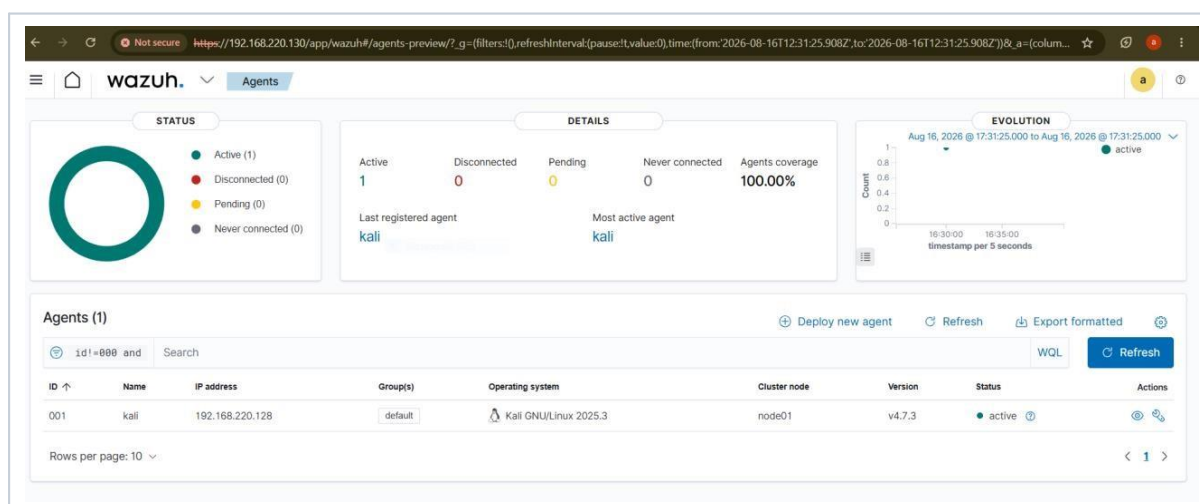


Fig. — Agents summary — Kali endpoint active and reporting

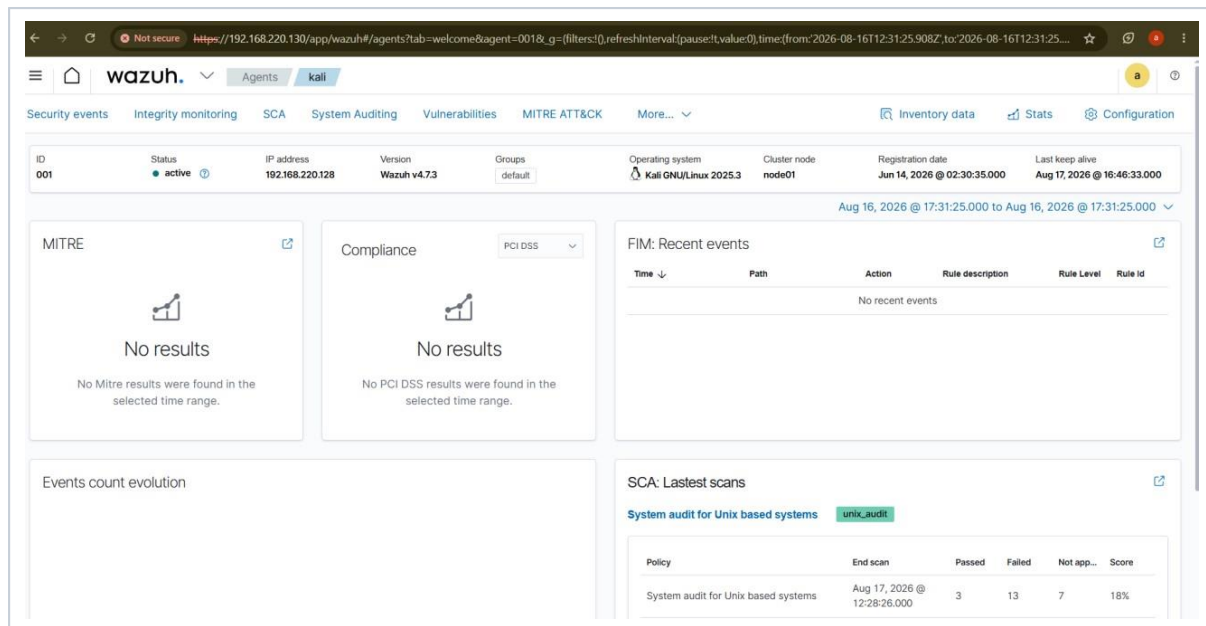


Fig. — Agent detail view — security events, integrity monitoring, SCA, compliance

17. Custom Decoder & Detection Rule Engineering

■ Custom Decoder Construction

An XML-based regex decoder was written inside `/var/ossec/etc/decoders/local_decoder.xml` to parse raw log strings into structured key-value fields (username, source IP), using `<prematch>` and `<regex>` tags to extract fields from application logs.

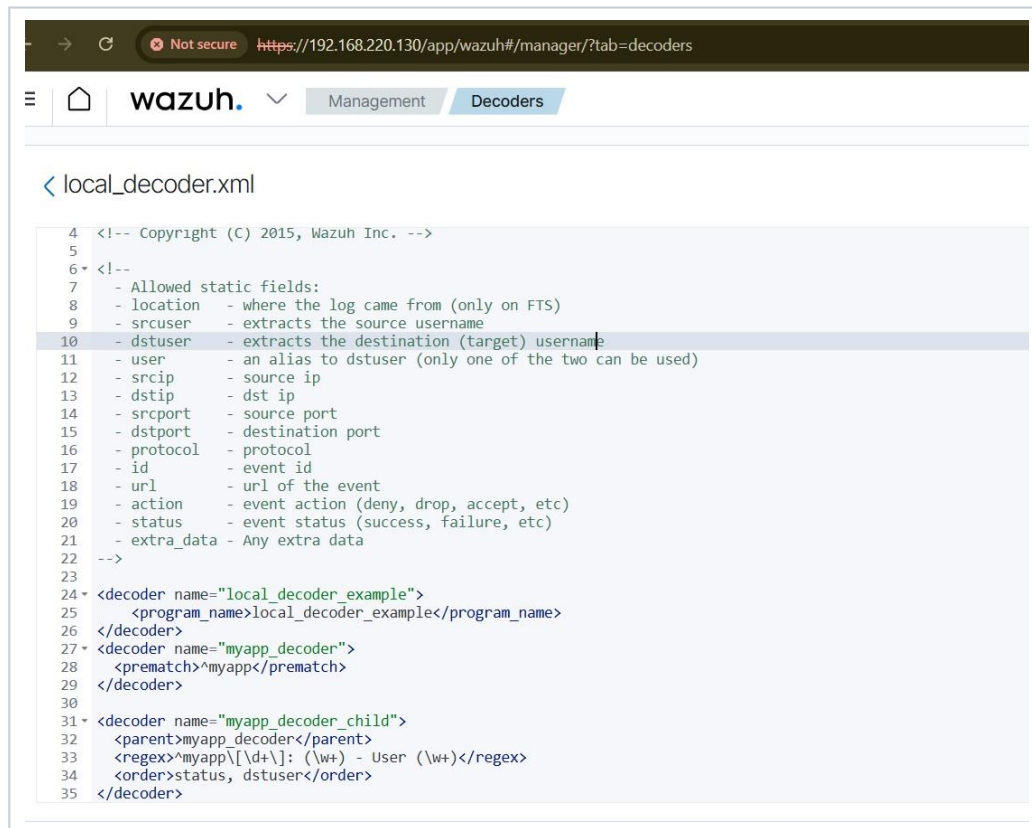


Fig. — `local_decoder.xml` — custom decoder definition

■ Detection Rule Development (Rule 10003)

Custom rule 10003 was configured inside `/var/ossec/etc/rules/local_rules.xml`, mapped to the MITRE ATT&CK framework, to generate severity-level alerts whenever failed login attempts occur — assigned severity level 7-10 and mapped to technique T1110 (Brute Force).

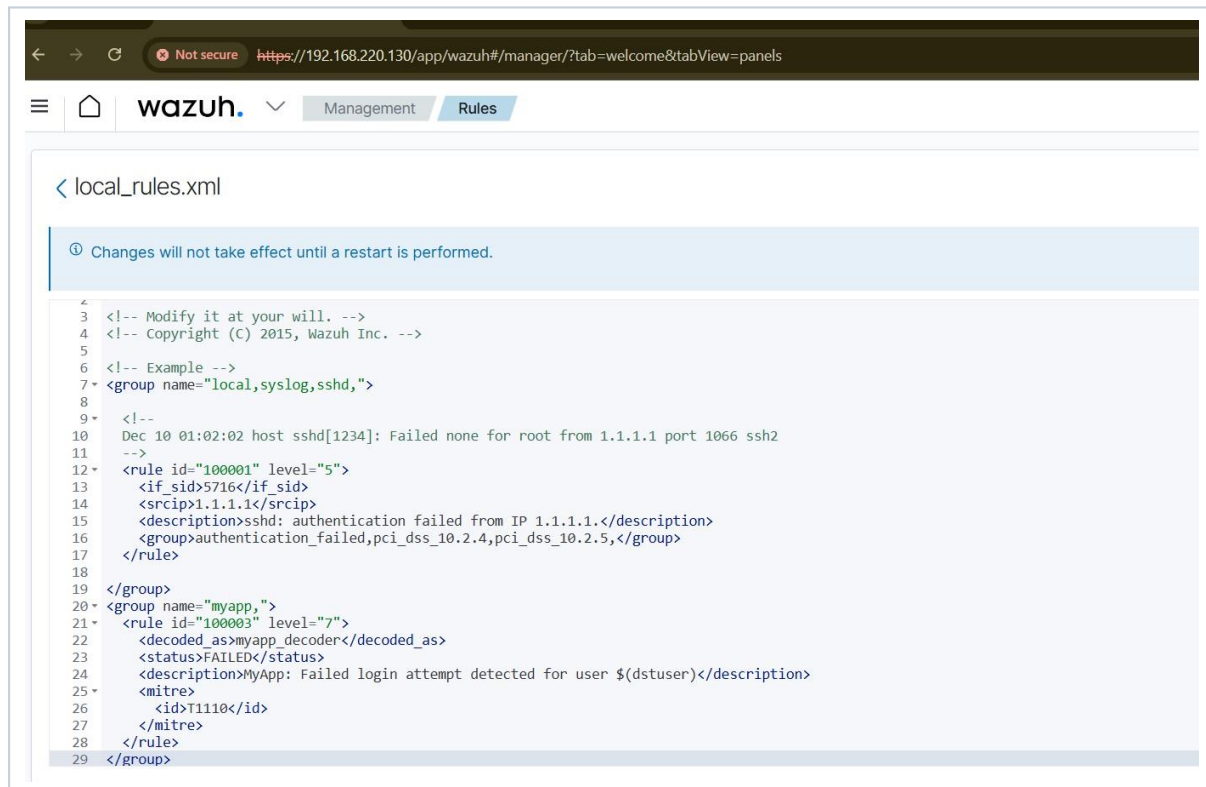


Fig. — `local_rules.xml` — custom detection rule mapped to MITRE T1110

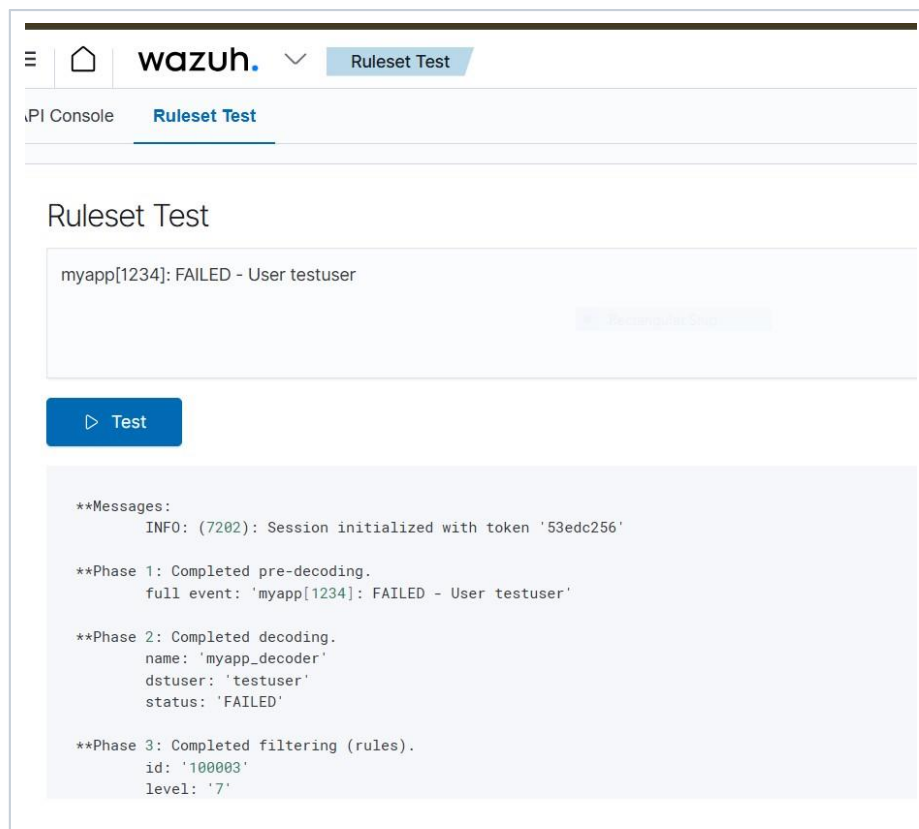


Fig. — Ruleset Test — validating decoder and rule output for a sample log line

18. Validation & Central Analysis

■ Offline Rule Engine Validation

Sample log lines were tested using `/var/ossec/bin/wazuh-logtest` to verify decoder extraction and rule evaluation without relying on live agent traffic, confirming Phase 1 (Pre-decoding), Phase 2 (Decoding), and Phase 3 (Rule Triggering).

```
Type one log per line

myapp[1234]: FAILED - User testuser

**Phase 1: Completed pre-decoding.
  full event: 'myapp[1234]: FAILED - User testuser'

**Phase 2: Completed decoding.
  name: 'myapp_decoder'
  dstuser: 'testuser'
  status: 'FAILED'

**Phase 3: Completed filtering (rules).
  id: '100003'
  level: '7'
  description: 'MyApp: Failed login attempt detected for user testuser'
  groups: ['myapp']
  fireddtimes: '1'
  mail: 'False'
  mitre.id: ['T1110']
  mitre.tactic: ['Credential Access']
  mitre.technique: ['Brute Force']
```

Fig. — wazuh-logtest CLI output — decoder and rule validated (Rule 100003, MITRE T1110)



Fig. — Alert field breakdown — agent, source, decoder, and MITRE mapping

■ Real-Time Alert Ingestion & Visual Dashboards

Live security events were analyzed and aggregated visualization widgets were built to provide centralized single-pane monitoring for security analysts. Events were filtered by rule.id: 10003 under Discover, saved as queries, and displayed as bar chart widgets on custom dashboards.

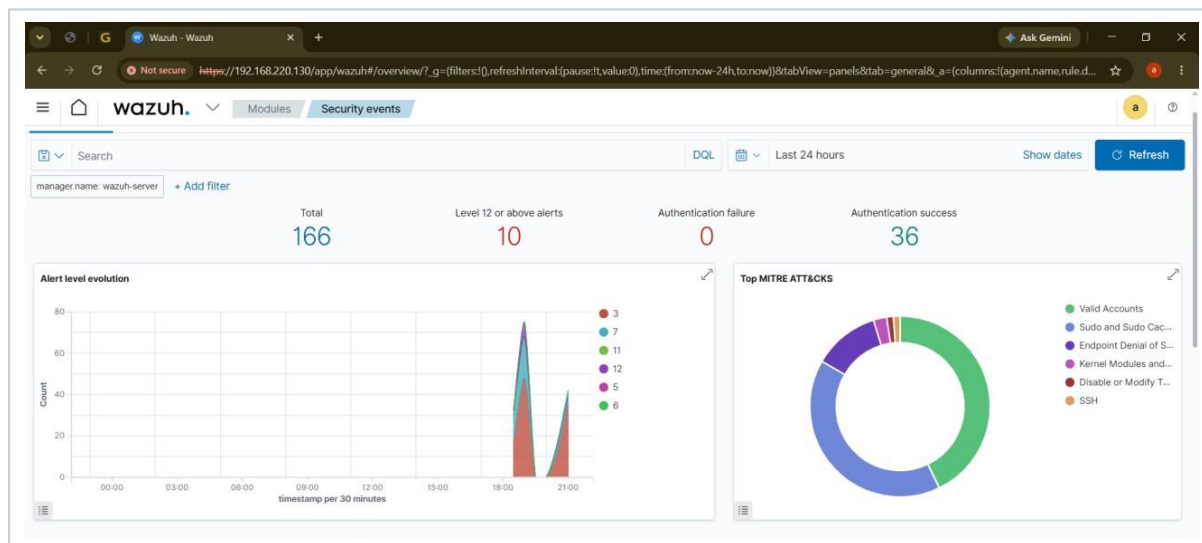
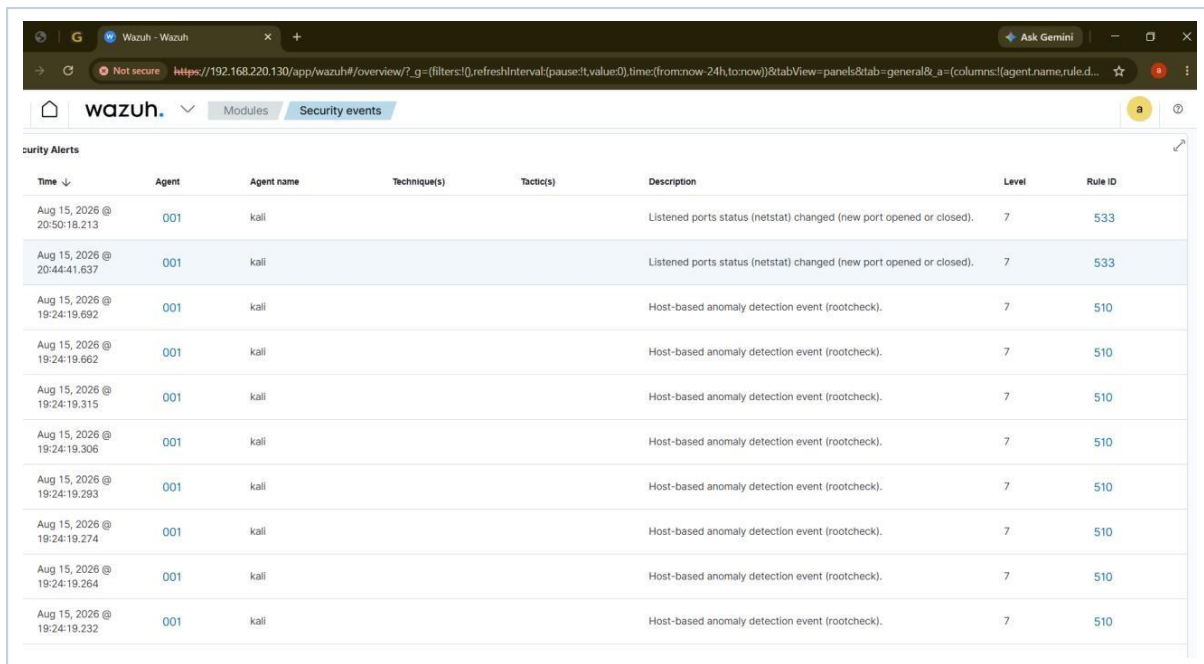


Fig. — Alert level evolution and Top MITRE ATT&CK; techniques — 24h view



Fig. — Top 5 agents and alert evolution trend — 24h view



The screenshot shows the Wazuh Security Alerts table. The table has columns for Time, Agent, Agent name, Technique(s), Tactic(s), Description, Level, and Rule ID. The data shows several alerts from agent 001 (kali) on August 15, 2026, including listened ports status changes and host-based anomaly detection events (rootcheck).

Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Aug 15, 2026 @ 20:50:18.213	001	kali			Listened ports status (netstat) changed (new port opened or closed).	7	533
Aug 15, 2026 @ 20:44:41.637	001	kali			Listened ports status (netstat) changed (new port opened or closed).	7	533
Aug 15, 2026 @ 19:24:19.692	001	kali			Host-based anomaly detection event (rootcheck).	7	510
Aug 15, 2026 @ 19:24:19.662	001	kali			Host-based anomaly detection event (rootcheck).	7	510
Aug 15, 2026 @ 19:24:19.315	001	kali			Host-based anomaly detection event (rootcheck).	7	510
Aug 15, 2026 @ 19:24:19.306	001	kali			Host-based anomaly detection event (rootcheck).	7	510
Aug 15, 2026 @ 19:24:19.293	001	kali			Host-based anomaly detection event (rootcheck).	7	510
Aug 15, 2026 @ 19:24:19.274	001	kali			Host-based anomaly detection event (rootcheck).	7	510
Aug 15, 2026 @ 19:24:19.264	001	kali			Host-based anomaly detection event (rootcheck).	7	510
Aug 15, 2026 @ 19:24:19.232	001	kali			Host-based anomaly detection event (rootcheck).	7	510

Fig. — Security Alerts table — FIM and host-based anomaly detection events

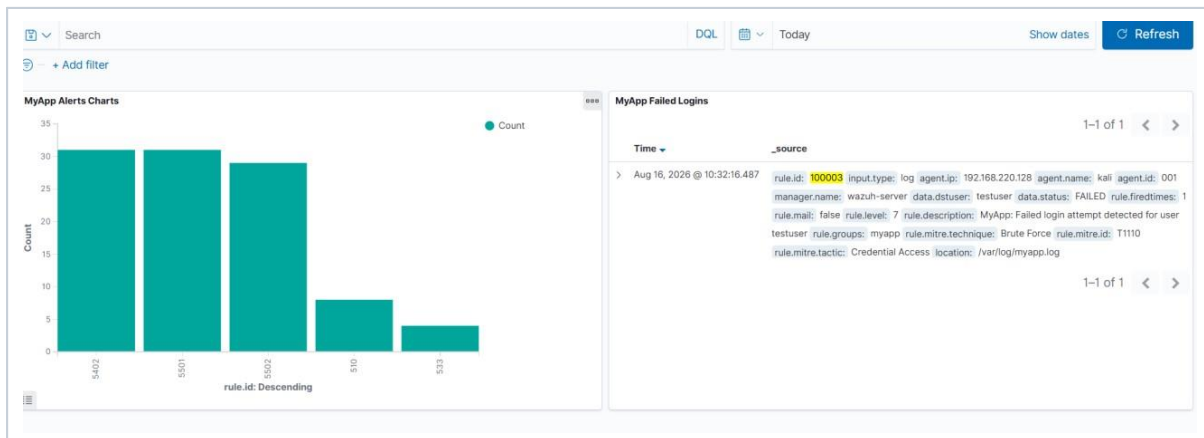
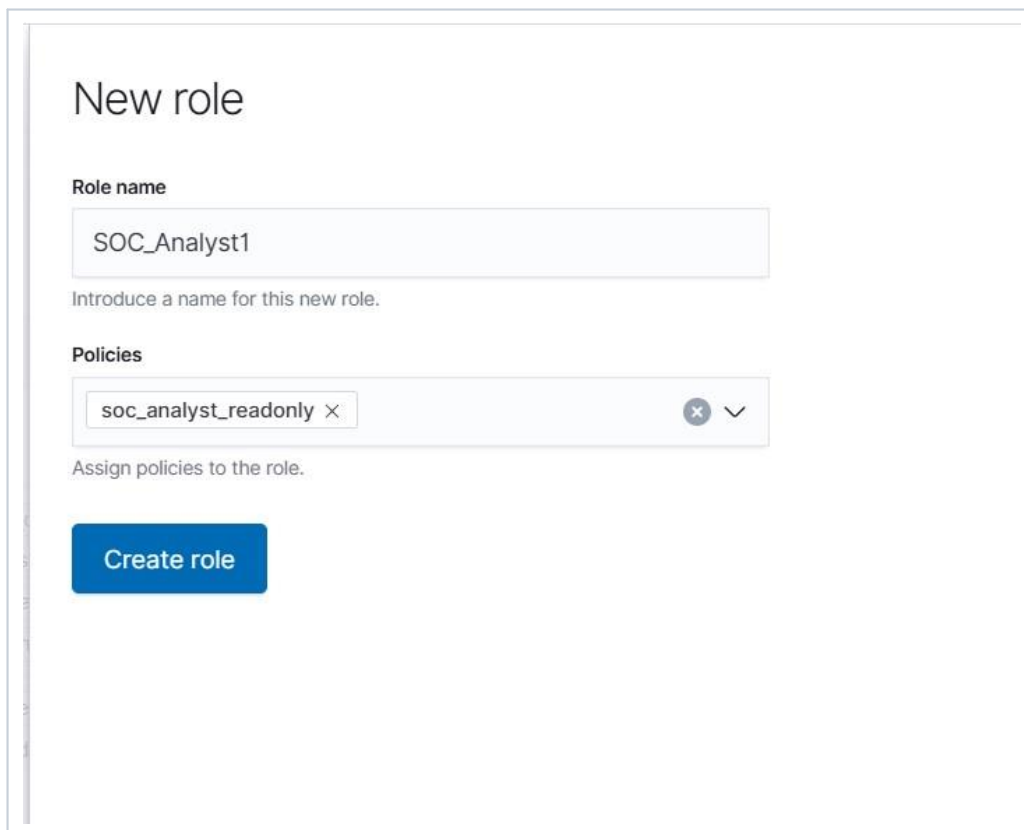


Fig. — Custom MyApp Alerts dashboard — failed login chart and event table

19. Role-Based Access Control (RBAC) Setup

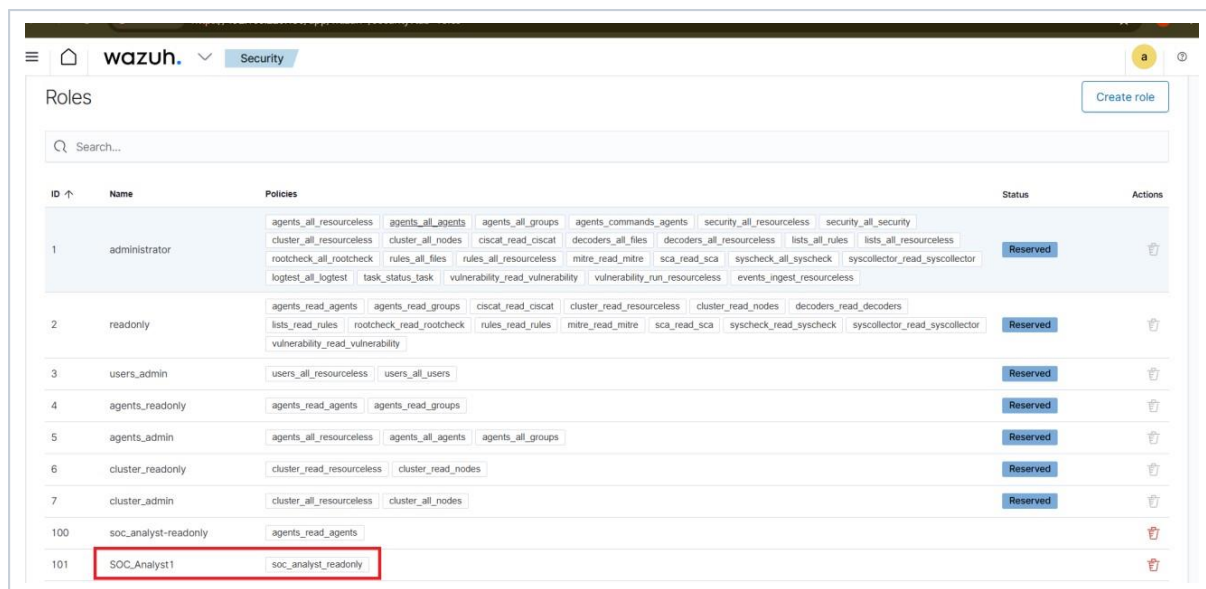
■ Defining a Read-Only Analyst Policy

A custom SOC_Analyst role was built with restricted read-only permissions to enforce the Principle of Least Privilege for security monitoring staff. Role policies were configured under Security > Roles, granting dashboard and discover read rights while withholding administrative privileges.



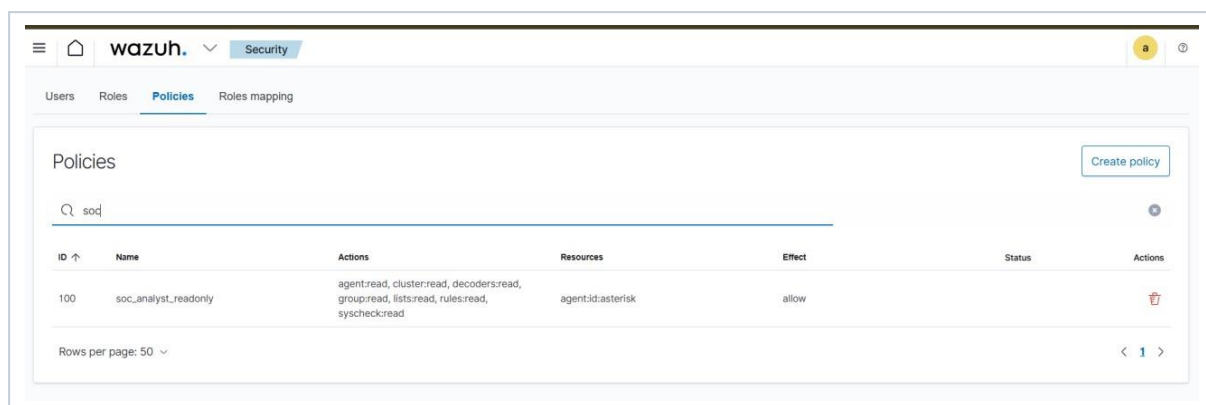
The screenshot shows the 'New role' configuration page. It has a title 'New role' at the top. Below it, there is a 'Role name' section with a text input field containing 'SOC_Analyst1' and a hint 'Introduce a name for this new role.' Below that is a 'Policies' section with a tag 'soc_analyst_readonly' and a hint 'Assign policies to the role.' At the bottom is a blue 'Create role' button.

Fig. — Creating the SOC_Analyst1 role with the soc_analyst_readonly policy



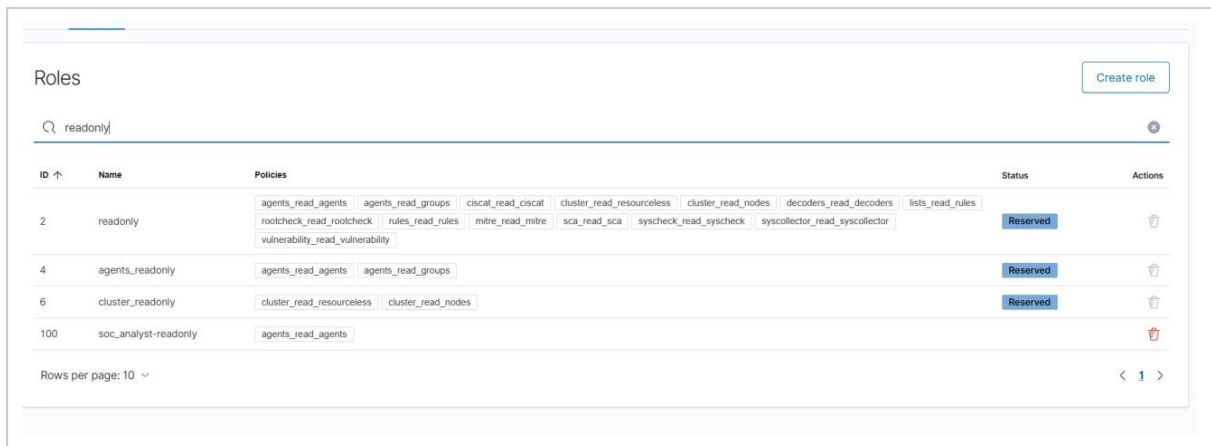
ID ↑	Name	Policies	Status	Actions
1	administrator	agents_all_resourceless agents_all_agents agents_all_groups agents_commands_agents security_all_resourceless security_all_security cluster_all_resourceless cluster_all_nodes ciscat_read_ciscat decoders_all_files decoders_all_resourceless lists_all_rules lists_all_resourceless rootcheck_all_rootcheck rules_all_files rules_all_resourceless mitre_read_mitre sca_read_sca syscheck_all_syscheck syscollector_read_syscollector logtest_all_logtest task_status_task vulnerability_read_vulnerability vulnerability_run_resourceless events_ingest_resourceless	Reserved	
2	readonly	agents_read_agents agents_read_groups ciscat_read_ciscat cluster_read_resourceless cluster_read_nodes decoders_read_decoders lists_read_rules rootcheck_read_rootcheck rules_read_rules mitre_read_mitre sca_read_sca syscheck_read_syscheck syscollector_read_syscollector vulnerability_read_vulnerability	Reserved	
3	users_admin	users_all_resourceless users_all_users	Reserved	
4	agents_readonly	agents_read_agents agents_read_groups	Reserved	
5	agents_admin	agents_all_resourceless agents_all_agents agents_all_groups	Reserved	
6	cluster_readonly	cluster_read_resourceless cluster_read_nodes	Reserved	
7	cluster_admin	cluster_all_resourceless cluster_all_nodes	Reserved	
100	soc_analyst-readonly	agents_read_agents		
101	SOC_Analyst1	soc_analyst_readonly		

Fig. — Roles list — SOC_Analyst1 added alongside default reserved roles



ID ↑	Name	Actions	Resources	Effect	Status	Actions
100	soc_analyst_readonly	agent.read, cluster.read, decoders.read, group.read, lists.read, rules.read, syscheck.read	agent.id:asterisk	allow		

Fig. — Policy definition — soc_analyst_readonly permissions



Roles

Create role

readonly

ID ↑	Name	Policies	Status	Actions
2	readonly	agents_read_agents agents_read_groups ciscat_read_ciscat cluster_read_resourceless cluster_read_nodes decoders_read_decoders lists_read_rules rootcheck_read_rootcheck rules_read_rules mitre_read_mitre sca_read_sca syscheck_read_syscheck syscollector_read_syscollector vulnerability_read_vulnerability	Reserved	
4	agents_readonly	agents_read_agents agents_read_groups	Reserved	
6	cluster_readonly	cluster_read_resourceless cluster_read_nodes	Reserved	
100	soc_analyst-readonly	agents_read_agents		

Rows per page: 10

< 1 >

Fig. — Roles filtered by 'readonly' — confirming least-privilege role separation

20. Conclusion

The end-to-end Wazuh SIEM deployment and security engineering workflow was successfully executed and validated across all project phases. Core platform services were verified, agent communication was established, and custom log ingestion was configured to expand endpoint visibility. The implementation of custom decoders and threat detection rules enabled precise log parsing and automated alerting mapped directly to the MITRE ATT&CK framework.

Offline testing via wazuh-logtest confirmed rule logic reliability, while central dashboards provided real-time visual analytics for security operations. Finally, enforcing Role-Based Access Control (RBAC) ensured that SOC analysts operate under strict least-privilege permissions, securing the platform's administrative functions.